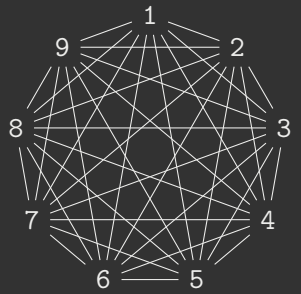
# >>> Quantum-Assisted Combinatorial Optimization for logistic and transportation: Emphasizing Analog Machines

Name: Samuel Deleplanque
deleplanque.samuel@gmail.com
Date: February 9, 2026



## >>> The course

* 3.5h Lecture (2 small breaks, inc. in eval.)
* 3.5h Lab (1 break, inc. in eval.)

# Quantum Annealing
## with D-Wave Machines

Practical lab

Sloths[1] moon colonization



>>> Outline


1. A Brief Introduction to Operations Research and Combinatorics
2. Quantum recaps and Adiabatic Machines
3. QUBO: an input for a quantum machine
4. Max-Cut problem
5. Traveling salesman Problem
   - Traveling salesman on a classical computer
   - QUBO of the Traveling Salesman Problem
   - To give a QUBO to the machine: addLinear + addQuadratic
6. Job-Shop Scheduling Problem (JSSP)
7. D-Wave hybrid solution for solving the CVRP

# A Brief Introduction to Operations Research

# Operations Research

# *Operations Research*

* Relates to the real-world processes and activities within an organization that *Operations Research* seeks to improve, such as logistics or inventory management.

# *Operations* *Research*

* Relates to the real-world processes and activities within an organization that *Operations Research* seeks to improve, such as logistics or inventory management.

* Highlights the discipline's analytical rigor through the study of methods to discover and implement problem solutions, often using mathematical models.

# Operations Research

* "Research on operations" seeks to organize activities mathematically: it starts by observing real problems, gathering data, and formulating them into models.

# Operations Research

* "Research on operations" seeks to organize activities mathematically: it starts by observing real problems, gathering data, and formulating them into models.
* Scientific model constructed to abstract essence of real problem; must capture essential characteristics.

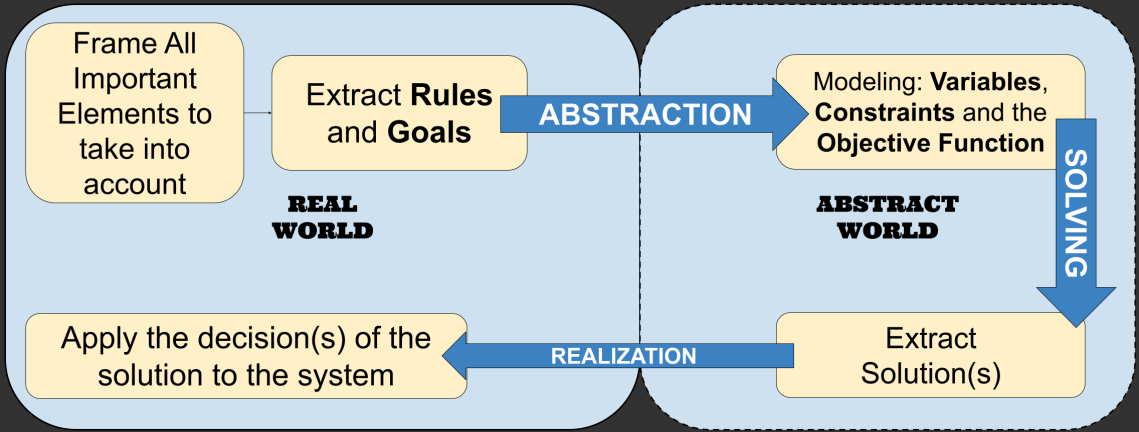
# Operations Research

* "Research on operations" seeks to organize activities mathematically: it starts by observing real problems, gathering data, and formulating them into models.
* Scientific model constructed to abstract essence of real problem; must capture essential characteristics.
* Operations Research often seeks optimal solution, which may not be unique.

# Operations Research

* "Research on operations" seeks to organize activities mathematically: it starts by observing real problems, gathering data, and formulating them into models.
* Scientific model constructed to abstract essence of real problem; must capture essential characteristics.
* Operations Research often seeks optimal solution, which may not be unique.
* Expertise includes mathematics, statistics, engineering, economics, management, computer science, physics, behavioral sciences, and specific OR techniques.

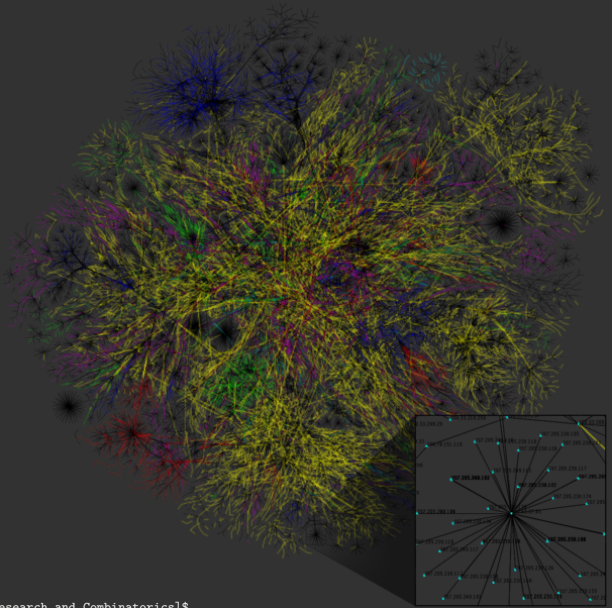
# >>> The stages of an operations research project



# >>> Applications of OR: Supply Chain Management



# >>> Applications of OR: Any network (here: internet map)



# >>> Applications of OR: Transportation Logistics



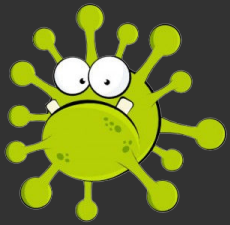
# >>> Applications of OR: Energy Management



# >>> Applications of OR: Humanitarian and Military



# >>> Applications of OR: Healthcare, the COVID crisis example



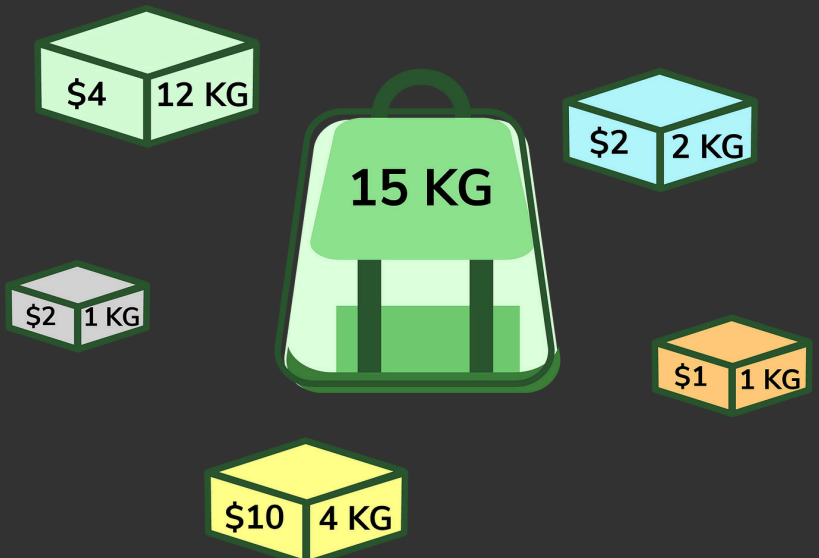
* Optimization of patient and medical equipment transport
* Supply of medical equipment
* Deployment of field hospitals
* Sizing and management of human resources
* Assignment and route calculation for home care staff
* Optimization of production (e.g., medicines and vaccines)
* Patient pathway planning

**Definition**

Combinatorial Explosion refers to the rapid growth of the number of possible solutions in certain combinatorial problems, making them difficult or impossible to solve exhaustively within a practical time frame.

>>> Knapsack Problem

Example: The Knapsack Problem



## >>> Combinatorial Explosion: Knapsack Problem

Example: The Knapsack Problem

Given a set of items, each with a weight and a value, determine the number of each item to include in a knapsack so that the total weight is less than or equal to a given limit and the total value is as large as possible.

Example: The Knapsack Problem

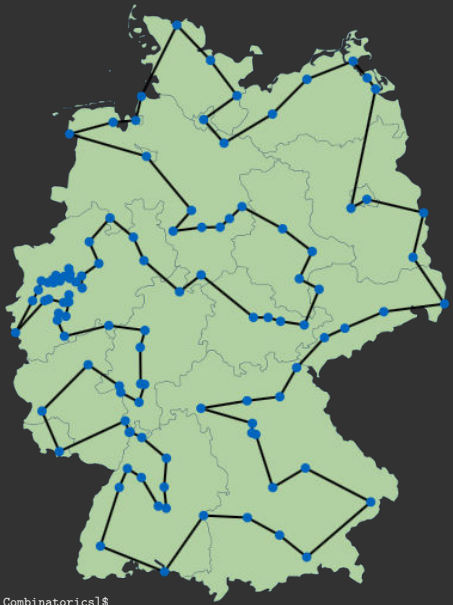
Given a set of items, each with a weight and a value, determine the number of each item to include in a knapsack so that the total weight is less than or equal to a given limit and the total value is as large as possible.

For $n$ items, there are $2^n$ possible combinations of items to consider.

* For 10 items: $2^{10}$ = 1,024 possible combinations
* For 20 items: $2^{20}$ = 1,048,576 possible combinations
* For 30 items: $2^{30} \approx$ 1.07 billion possible combinations

As the number of items increases, the problem becomes intractable for exact solution methods due to the combinatorial explosion.

# >>> Combinatorial Explosion: Traveling Salesman Problem



## >>> Combinatorial Explosion: Traveling Salesman Problem and Location Problem

Example: The Traveling Salesman Problem (TSP)

Given a list of cities, the goal is to determine the shortest possible route that visits each city exactly once and returns to the origin city.

## >>> Combinatorial Explosion: Traveling Salesman Problem and Location Problem

Example: The Traveling Salesman Problem (TSP)

Given a list of cities, the goal is to determine the shortest possible route that visits each city exactly once and returns to the origin city.

For $n$ cities, there are $(n-1)!$ possible routes to consider.

* For 10 cities: $9!$ = 362,880 possible routes
* For 15 cities: $14!$ = 87,178,291,200 possible routes
* For 20 cities: $19! \approx$ 1.216451e+17 possible routes

## >>> Combinatorial Explosion: Traveling Salesman Problem and Location Problem

Example: The Traveling Salesman Problem (TSP)
Given a list of cities, the goal is to determine the shortest possible route that visits each city exactly once and returns to the origin city.

For $n$ cities, there are $(n-1)!$ possible routes to consider.

* For 10 cities: $9!$ = 362,880 possible routes
* For 15 cities: $14!$ = 87,178,291,200 possible routes
* For 20 cities: $19! \approx$ 1.216451e+17 possible routes

Example: Location Problem: we want to locate 10 data centers in a city...

* 10 centers out of 12 centers: 66 solutions
* 10 out of 30: 30 million solutions
* 10 out of 100: billions of billions of solutions

## >>> Combinatorial Explosion: Traveling Salesman Problem and Location Problem

Example: The Traveling Salesman Problem (TSP)
Given a list of cities, the goal is to determine the shortest possible route that visits each city exactly once and returns to the origin city.

For $n$ cities, there are $(n-1)!$ possible routes to consider.

* For 10 cities: $9!$ = 362,880 possible routes
* For 15 cities: $14!$ = 87,178,291,200 possible routes
* For 20 cities: $19! \approx$ 1.216451e+17 possible routes

Example: Location Problem: we want to locate 10 data centers in a city...

* 10 centers out of 12 centers: 66 solutions
* 10 out of 30: 30 million solutions
* 10 out of 100: billions of billions of solutions

Keep in mind...

* $10^{57}$ atoms in the solar system.... $8.10^{67}$ ways to sort a deck of 52 cards
* The same to schedule 52 tasks...

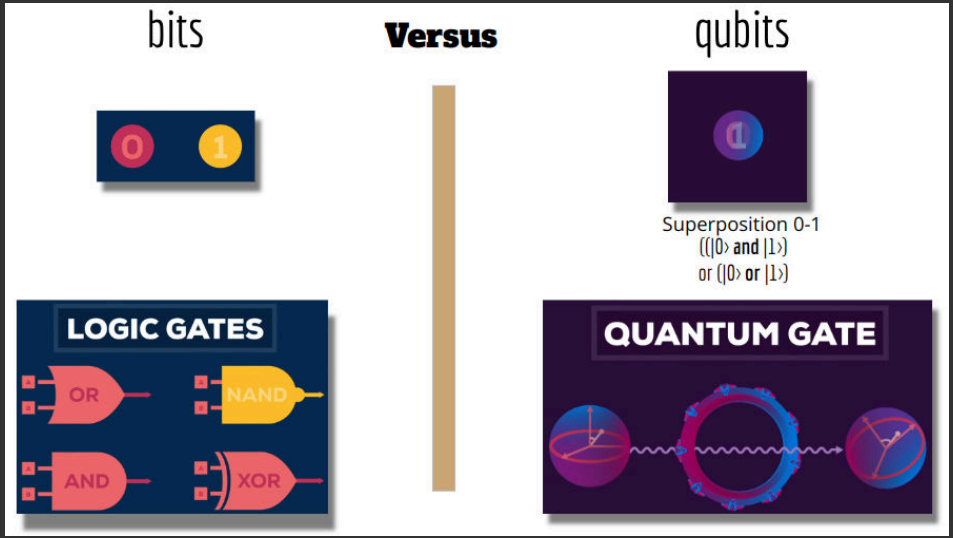
# MILP

$$\min \quad \sum_{r \in R} z_{n_r} \tag{1}$$

s.t.

$$x_e^{end} = x_{e+1}^{begin}, \quad e \in RE_r, r \in R : e = 1..n_r - 1, \tag{2}$$
$$x_e^{end} \geq x_e^{begin} + d_e, \quad e \in E, \tag{3}$$
$$x_e^{begin} \geq \tau_e^{begin}, \quad e \in E : i_e = 1, \tag{4}$$
$$x_e^{end} - \tau_e^{end} \leq z_e, \quad e \in E, \tag{5}$$
$$\sum_{t \in T_s} q_{et} = 1, \quad e \in SE_s, s \in S, \tag{6}$$
$$q_{et} + q_{\hat{e}t} - 1 \leq \lambda_{e\hat{e}} + \gamma_{e\hat{e}}, \quad e, \hat{e} \in SE_s, t \in T_s, s \in S : e < \hat{e}, \tag{7}$$
$$x_{\hat{e}}^{begin} - x_e^{end} \geq \Delta_s^M \gamma_{e\hat{e}} - M(1 - \gamma_{e\hat{e}}), e < \hat{e} \text{ in } SE_s, s \in S, o_e \neq o_{\hat{e}}, \tag{8}$$
$$x_{\hat{e}}^{begin} - x_e^{end} \geq \Delta_s^F \gamma_{e\hat{e}} - M(1 - \gamma_{e\hat{e}}), e < \hat{e} \text{ in } SE_s, s \in S, o_e = o_{\hat{e}}, \tag{9}$$
$$x_e^{begin} - x_{\hat{e}}^{end} \geq \Delta_s^M \lambda_{e\hat{e}} - M(1 - \lambda_{e\hat{e}}), e < \hat{e} \text{ in } SE_s, s \in S, o_e \neq o_{\hat{e}}, \tag{10}$$
$$x_e^{begin} - x_{\hat{e}}^{end} \geq \Delta_s^F \lambda_{e\hat{e}} - M(1 - \lambda_{e\hat{e}}), e < \hat{e} \text{ in } SE_s, s \in S, o_e = o_{\hat{e}}, \tag{11}$$
$$\lambda_{e\hat{e}} + \gamma_{e\hat{e}} \leq 1, \quad e, \hat{e} \in SE_s, s \in S : e < \hat{e}, \tag{12}$$
$$x_e^{begin} \geq w_t^{end} q_{et} - M\alpha_e^{w_t}, \quad e \in SE_s, t \in T_s, s \in S, \tag{13}$$
$$x_e^{end} \leq w_t^{begin} q_{et} + M(1 - \alpha_e^{w_t}), \quad e \in SE_s, t \in T_s, s \in S, \tag{14}$$
$$x_e^{begin}, x_e^{end}, z_e \geq 0, \quad e \in E, \tag{15}$$
$$\gamma_{e\hat{e}}, \lambda_{e\hat{e}}, \in \{0,1\}, \quad e, \hat{e} \in SE_s, s \in S : e < \hat{e}, \tag{16}$$
$$q_{et}, \alpha_e^{w_t} \in \{0,1\}, \quad e \in SE_s, t \in T_s, s \in S, \tag{17}$$

# Quantum recaps
# and
# Adiabatic Machines




Pic source: In a Nut Shell

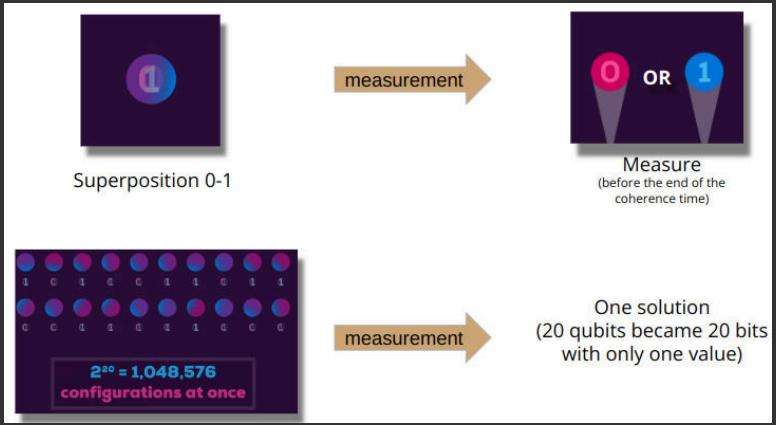



Figure 1: The measurement of qubits is the process of determining the state of a qubit, collapsing its superposition of states into one of the basis states, according to the probability amplitudes defined by the qubit's quantum state before measurement.

## >>> Senile qubits...

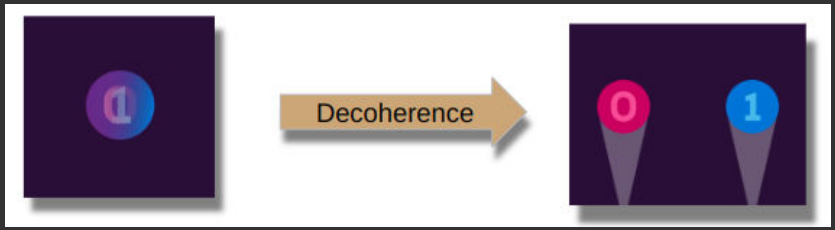



Figure 2: Example on a qubit: break the superposition state to 1 or 0.

* Decoherence occurs due to interactions with environmental factors such as radiation, light, sound, magnetic fields, etc
* The system goes from a quantum system to a classical one
* Limit the time to work on qubits

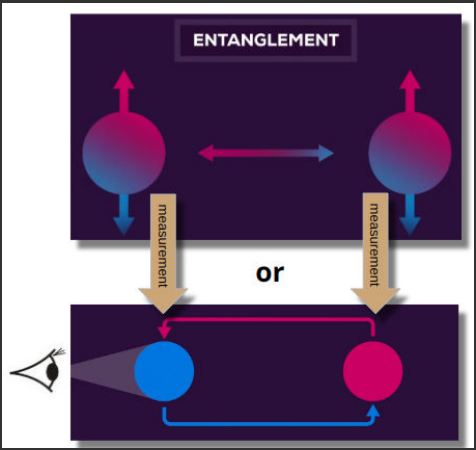



Figure 3: Entanglement is a quantum phenomenon where two or more particles become linked in such a way that the state of one particle instantaneously influences the state of the other, regardless of the distance separating them.

## >>> Universal gate-based quantum computers

* Overview: Universal gate-based Quantum Computers use quantum gates to perform operations on qubits. They are programmable and can perform a wide range of quantum algorithms.
* User Inputs: Quantum circuit consisting of a sequence of quantum gates.

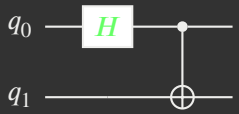



Figure 4: One entanglement State (Hadamard gate and CNOT).

## >>> Universal gate-based quantum computers

* Overview: Universal gate-based Quantum Computers use quantum gates to perform operations on qubits. They are programmable and can perform a wide range of quantum algorithms.
* User Inputs: Quantum circuit consisting of a sequence of quantum gates.

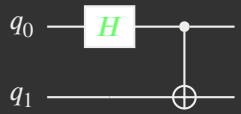
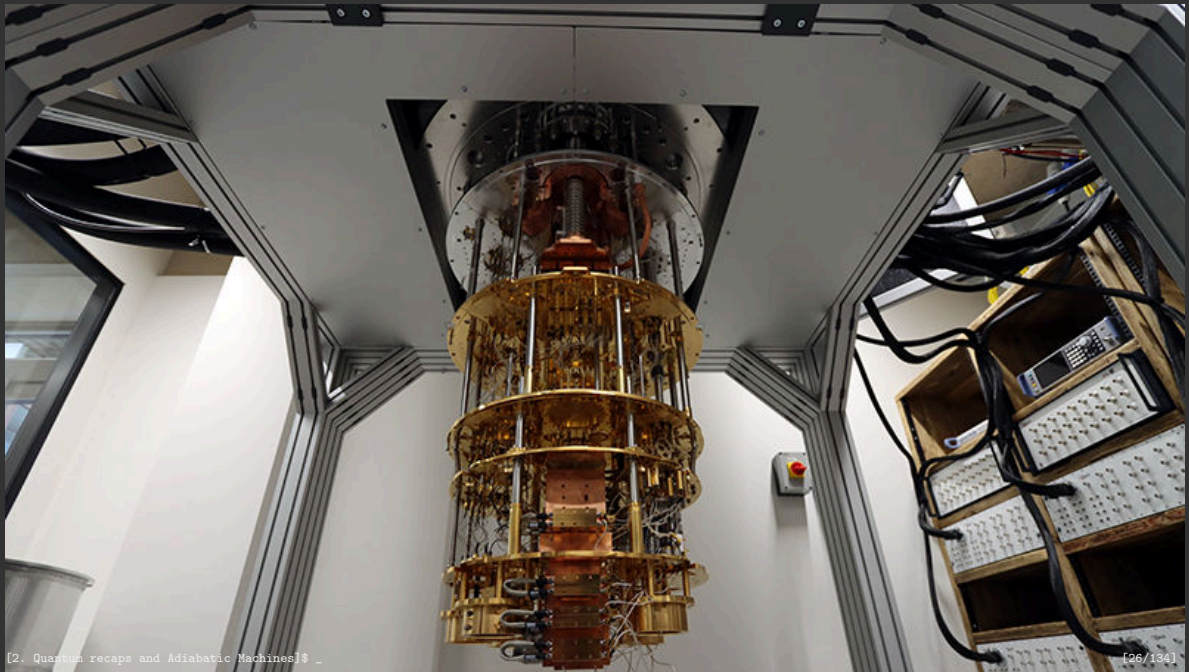



Figure 4: One entanglement State (Hadamard gate and CNOT).

* Pro: Versatile (or any quantum algorithm)
* Cons: Require error correction and fault-tolerant mechanisms (decoherence pbm)
* Notable Brands: IBM (1k qubits), Atom Computing -?- (1k), Google (70))
* Well known quantum algorithms: Grover [Grover, 1996], Shor [Shor, 1997], QAOA ([Farhi et al., 2014]) etc.



## >>> Analog Quantum Computers

* Overview: Analog Quantum Computers are specially designed to solve specific types of problems, such as optimization problems
* User Inputs: Problem-specific Hamiltonian that encodes the problem to be solved (e.g., Ising model)

## >>> Analog Quantum Computers

* Overview: Analog Quantum Computers are specially designed to solve specific types of problems, such as optimization problems
* User Inputs: Problem-specific Hamiltonian that encodes the problem to be solved (e.g., Ising model)
* Pro: Potentially more robust to errors and suited for solving optimization problems
* Cons: Less versatile, tailored to specific problems
* Notable Brands: D-Wave (5k qubits), Pasqal (1k qubits)
* Quantum Annealing (QA) or other adiabatic quantum computations

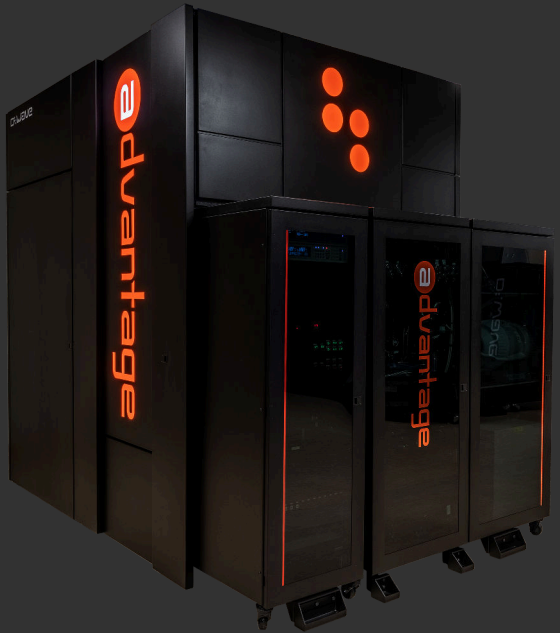
## >>> Technologies in D-Wave and Pasqal machines

### D-Wave

* Qubits: electron's spin (nodium)
* $|0\rangle$ and $|1\rangle$ states: spin $\uparrow$ and $\downarrow$
* Transition state: programmable magnetic flux biases
* "Entanglement": tunable inductive couplers (Josephson junction)
* Measure: magnetic flux generated by the current states (SQUIDs)
* INPUT: Ising or QUBO models
* Solving technique: Quantum Annealing

### Pasqal

* Qubits: neutral atom (rubidium)
* $|0\rangle$ and $|1\rangle$ states: ground state and Rydberg state, resp
* Transition state: laser Beams
* Entanglement: Rydberg blockade
* Measure: fluorescence imaging
* INPUT: Ising or QUBO models (or series of gates -soon)
* Solving technique: quantum adiabatic algorithm (and more)


D-WAVE
advantage
advantage
D-WAVE

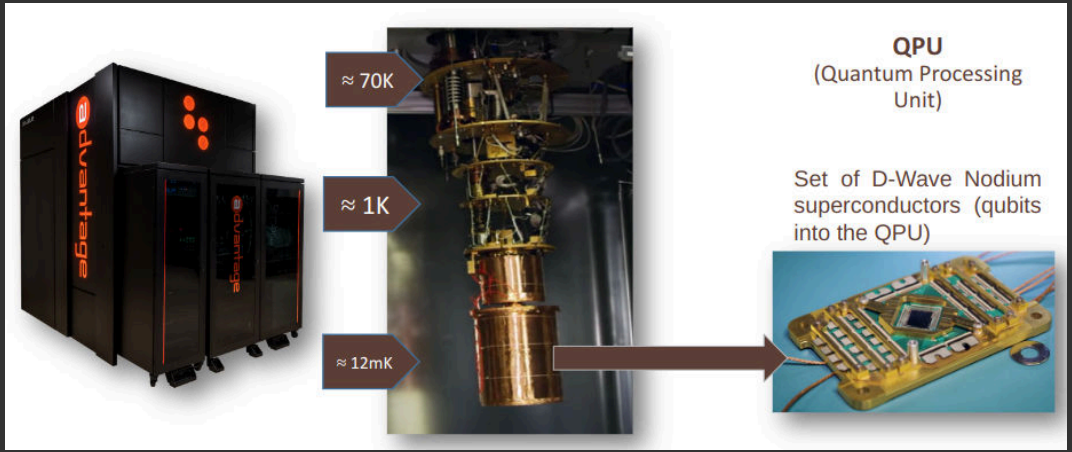



Figure 7: Inside the 5k qubits Advantage Machine (D-Wave).

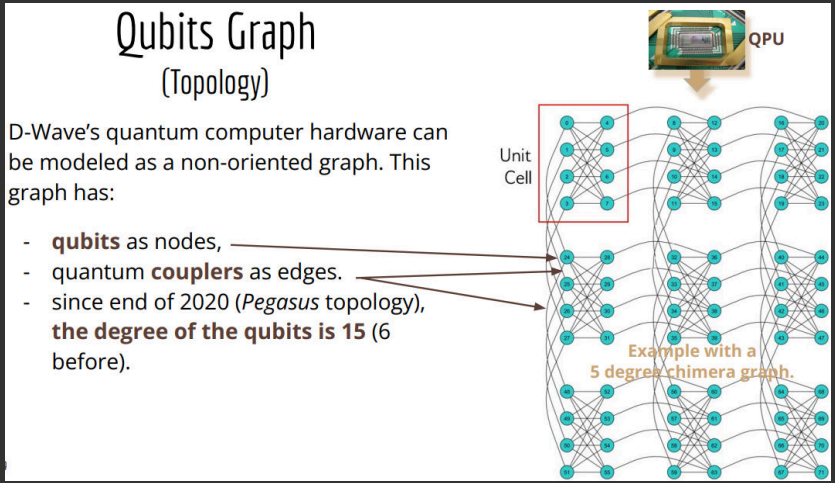



Figure 8: Static Topology of the qubit graph. An old example: the chimera graph.

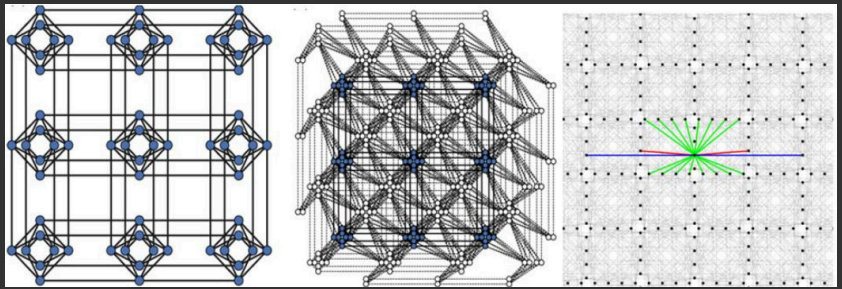


Figure 9: Chimera, Pegasus and Zephir topologies, resp..

| Topology | Chimera | Pegasus | Zephyr |
|---|---|---|---|
| Name Machine | DW_2000Q | Advantage_System | Adv.2_proto. |
| Machine Version | 6 | 6.4 | 2.3 |
| \|Qubits\| | 2041 | 5616 | 1200 |
| Connectivity | 6 | 15 | 20 |

Table 1: Characteristics of the 3 main topologies.

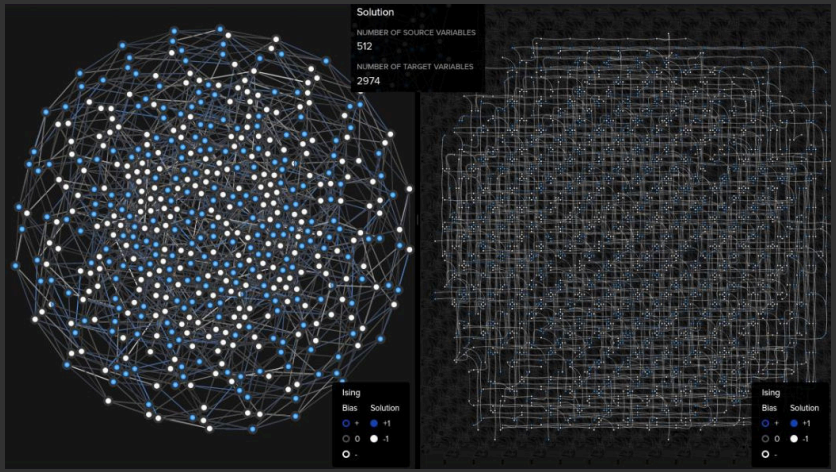
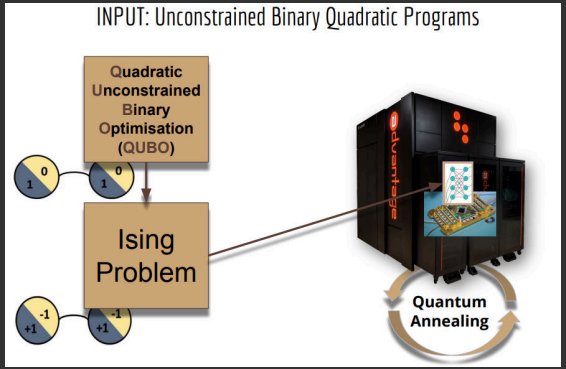
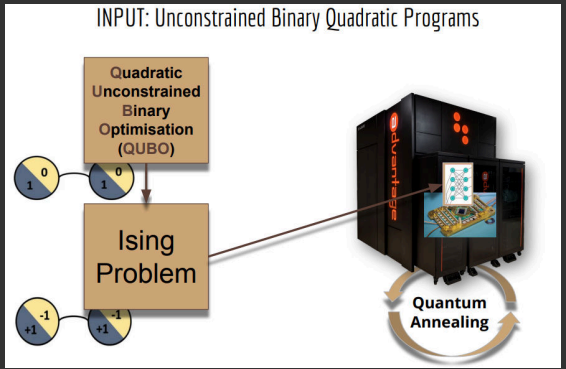



Figure 10: A sparse graph example: from 512 vertices of a Max-Cut instance (left) to 2974 qubits in the qubits graph (right). sg3dl051000 Max-Cut instance from [Festa et al., 2002]. Topology: Pegasus.


INPUT: Unconstrained Binary Quadratic Programs
Quadratic
Unconstrained
Binary
Optimisation
(QUBO)
0
1
0
1
Ising
Problem
-1
+1
-1
+1
Quantum
Annealing

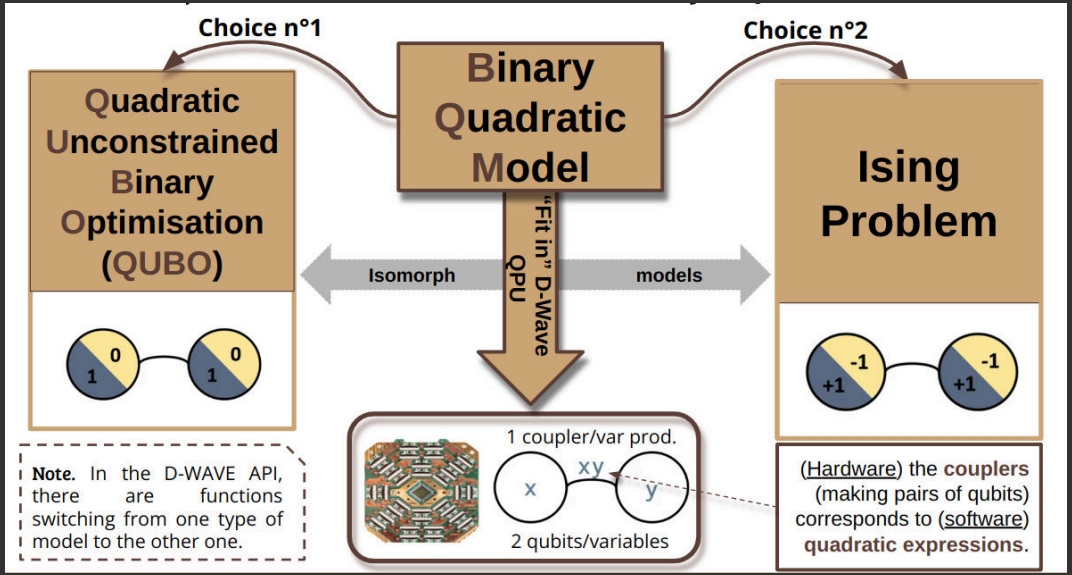



## Ising

* Overview: Used in statistical mechanics.
* Formulation: Involves spins $s$ that can be in one of two states: up (+1) or down (-1).
* $H_{Ising} = \sum_i h_i s_i + \sum_{i<j} J_{i,j} s_i s_j.$

## QUBO (Quadratic Unconstrained Bin. Opti.)

* Overview: Used in optimization.
* Formulation: Variables $x$ take binary values (0 or 1), all constraints relaxed (weighted penalties).
* $f_{QUBO}(x) = \sum_i Q_{i,i} x_i + \sum_{i<j} Q_{i,j} x_i x_j = x^T Q x.$


Choice n°1
Choice n°2
Binary Quadratic Model
Quadratic Unconstrained Binary Optimisation (QUBO)
Ising Problem
Isomorph
models
"Fit in" D-Wave QPU
0
1
0
1
-1
+1
-1
+1
1 coupler/var prod.
xy
x
y
2 qubits/variables
Note. In the D-WAVE API, there are functions switching from one type of model to the other one.
(Hardware) the couplers (making pairs of qubits) corresponds to (software) quadratic expressions.

# Quantum Annealing



Hidetoshi Nishimori

- Father of the **Quantum Annealing Metaheuristic**
  - Works on Statistical physics,
  - Paper: Kadowaki, Tadashi, and Hidetoshi Nishimori. "Quantum annealing in the transverse Ising model." *Physical Review E* 58.5 published in **1998;**
- Based on classical computer Metaheuristic Simulated Annealing
  - Kirkpatrick, Scott, C. Daniel Gelatt, and Mario P. Vecchi. "Optimization by simulated annealing." *science* 220.4598 (1983): 671-680;
- Prove that the Quantum Annealing gives better results that the Simulated Annealing,
- A good conference from him: https://www.youtube.com/watch?v=OQ91L96YWCk.

---

D:Wave
The Quantum Computing Company™

- First (hardware) implementation of the Quantum Annealing,
- Starting the work in **1999**, the 1st computer based on quantum annealing released in **2007**.

# Quantum annealing *(fr : recuit quantique)* & Optimization Problems

"Physics can help solve [Optimization Problems] because we can frame them as energy minimization problems. "

"The D-Wave system can be viewed as a **hardware heuristic** that **minimizes** [..] **objective functions** using a physically realized version of quantum annealing."

INPUT --- Quantum Annealing --- OUTPUT

The very basic idea

First Step

**The qubits start in a superposition** state (“**zero and one**”).

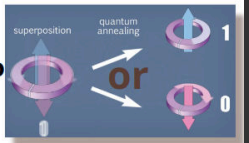


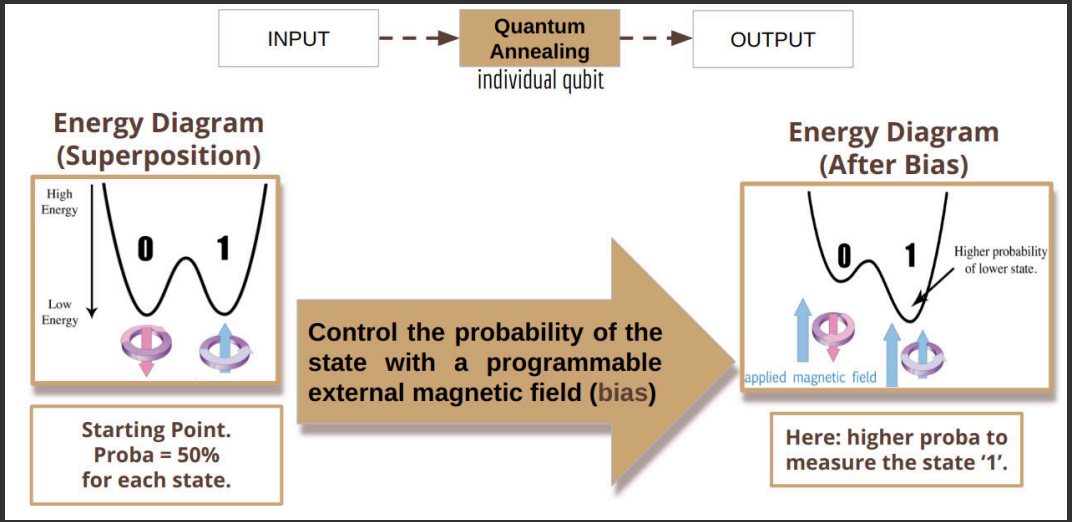

and

**[...]**
**“Work on the 0 and 1 states (amplitudes).”**
**[...]**

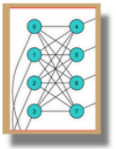
Last step

The measure of each qubit make them going to either the **zero state** or the **one state**.

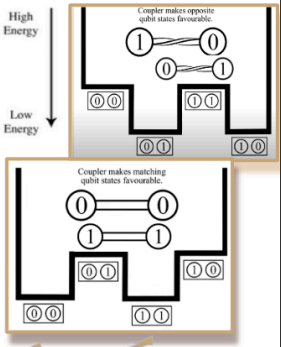



INPUT
Quantum Annealing
individual qubit
OUTPUT
Energy Diagram (Superposition)
High Energy
Low Energy
0
1
Starting Point. Proba = 50% for each state.
Control the probability of the state with a programmable external magnetic field (bias)
Energy Diagram (After Bias)
0
1
Higher probability of lower state.
applied magnetic field
Here: higher proba to measure the state '1'.

# ""Entanglement"" of pair of qubits



- **The "real power" of such a system appears once we start to link the qubits together.**
  - by **couplers** (using **entanglement** behaviour).
    - **The couplers define how a pair of qubits ends up, in the same or in opposite state**, (0&0, 1&1 or 1&0, 0&1).




- **Once 2 qubits are "entangled", they can be considered as a single object, but with 4 states.**
  - When we consider a pair of qubits to be the same (or to be opposite) **the machine will "physically" make the equality (or the inequality)** "**energetically favorable**", i.e., to "low the energy" of those states.

This is not a strict entanglement, it is a matter of probability.

# the Quantum Annealing process

(already implemented in D-Wave machines)

**Initialise**. Qubits are placed in an initial superposition state according to the Hamiltonian $H_i$.

**Anneal**. The quantum particle process makes a transition from initial Hamiltonian $H_i$ to the Hamiltonian $H_f$ adiabatically (if possible).

**Read**. Qubits values are read, yielding a solution S.



**Resampling**. Since any quantum computation is probabilistic, there is a positive probability that the computation does not finish in ground state (i.e., not yielding an optimal solution), so we iterate.

# QUBO Models

* Such model is a mathematical construction used to represent significant aspects of real-world problems.

* Such model is a mathematical construction used to represent significant aspects of real-world problems.
* Three main components in an optimization model:
  1. Variables: Represent modifiable components creating different configurations.

* Such model is a mathematical construction used to represent significant aspects of real-world problems.
* Three main components in an optimization model:
  1. Variables: Represent modifiable components creating different configurations.
  2. Constraints: Represent limitations on variables.

* Such model is a mathematical construction used to represent significant aspects of real-world problems.
* Three main components in an optimization model:
  1. Variables: Represent modifiable components creating different configurations.
  2. Constraints: Represent limitations on variables.
  3. Objective function: Assigns values to configurations; goal is optimization (minimizing or maximizing the Objective function).

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.
* A model with binary variables CANNOT be solved in a short time.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.
* A model with binary variables CANNOT be solved in a short time.
* => Discrete optimization

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.
* A model with binary variables CANNOT be solved in a short time.
* => Discrete optimization

## Integer Variables

* Units Produced: Manufacture 100 units of a product.
* Assignments: Assign 3 tasks to an employee.
* Trips: 4 delivery trips per day.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.
* A model with binary variables CANNOT be solved in a short time.
* => Discrete optimization

## Integer Variables

* Units Produced: Manufacture 100 units of a product.
* Assignments: Assign 3 tasks to an employee.
* Trips: 4 delivery trips per day.
* A model with integer variables CANNOT be solved in a short time.

# >>> Types of Decision Variables

## Continuous Variables

* Time: 3.5 hours spent on a task.
* Temp': Maintain a room at 22.5°C.
* Concentration: 7.8% salt solution.
* A model with (only) continuous (real) variables can be solved in a short time.
* => Continuous optimization

## Binary Variables

* Switch: ON (1) or OFF (0).
* Decision: Accept (1) or Reject (0) a proposal.
* State: Open (1) or Closed (0) status of a gate.
* A model with binary variables CANNOT be solved in a short time.
* => Discrete optimization

## Integer Variables

* Units Produced: Manufacture 100 units of a product.
* Assignments: Assign 3 tasks to an employee.
* Trips: 4 delivery trips per day.
* A model with integer variables CANNOT be solved in a short time.
* => Discrete optimization

## Lower or Equal Constraints ($\leq$)

* Budget Limitation: Can't spend more than $X$.
* Maximum Capacity: A venue can host $\leq 500$ people.
* Speed Limit: Vehicles can't drive faster than 60 mph.
* Storage Limit: A warehouse can store $\leq 1000$ boxes.

# >>> Types of Constraints

## Lower or Equal Constraints ($\leq$)

* Budget Limitation: Can't spend more than $X$.
* Maximum Capacity: A venue can host $\leq$ 500 people.
* Speed Limit: Vehicles can't drive faster than 60 mph.
* Storage Limit: A warehouse can store $\leq$ 1000 boxes.

## Equality Constraints ($=$)

* Balance: Inputs = Outputs in a system.
* Reservation: Book exactly 10 seats.
* Requirement: Produce = Demand.
* Allocation: Each person gets 1 ticket.

# >>> Types of Constraints

## Lower or Equal Constraints ($\leq$)

* Budget Limitation: Can't spend more than $X$.
* Maximum Capacity: A venue can host $\leq$ 500 people.
* Speed Limit: Vehicles can't drive faster than 60 mph.
* Storage Limit: A warehouse can store $\leq$ 1000 boxes.

## Equality Constraints ($=$)

* Balance: Inputs = Outputs in a system.
* Reservation: Book exactly 10 seats.
* Requirement: Produce = Demand.
* Allocation: Each person gets 1 ticket.

## Greater or Equal Constraints ($\geq$)

* Minimum Production: Produce $\geq$ 100 units.
* Safety Standards: Maintain a reserve $\geq$ 50 liters.
* Minimum Attendance: At least 30 students in a class.
* Quorum: A meeting needs $\geq$ 10 members to proceed.

# >>> Modeling. Objective Function: Cost vs. Profit

## Cost

* Energy: The amount of electricity consumed by a device.
* Distances: Fuel consumed due to a long commute.
* Time: Hours spent on a non-productive activity.
* ...

Anything we would want to minimize.

# >>> Modeling. Objective Function: Cost vs. Profit

## Cost

* Energy: The amount of electricity consumed by a device.
* Distances: Fuel consumed due to a long commute.
* Time: Hours spent on a non-productive activity.
* ...

Anything we would want to minimize.

## Profit

* Revenue: Money earned from selling a product.
* Yield: Quantity of crops produced per acre.
* Investment Returns: Profit from stocks or bonds.
* ...

Anything we would want to maximize.

# >>> What are QUBO Models ?

* QUBO (Quadratic Unconstrained Binary Optimization) models are optimization models where constraints are relaxed. Here, no constraints, just an Objective Function based on binaries variables:

$$* \min_{x \in \{0,1\}^n} \left( \sum_{i=1}^{n} Q_{ii} x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} Q_{ij} x_i x_j \right)$$

## >>> What are QUBO Models ?

* QUBO (Quadratic Unconstrained Binary Optimization) models are optimization models where constraints are relaxed. Here, no constraints, just an Objective Function based on binaries variables:

$$* \min_{x \in \{0,1\}^n} \left( \sum_{i=1}^{n} Q_{ii} x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} Q_{ij} x_i x_j \right)$$

* Historically, these models were solved by a few classical solvers.

* QUBO (Quadratic Unconstrained Binary Optimization) models are optimization models where constraints are relaxed. Here, no constraints, just an Objective Function based on binaries variables:

$$* \min_{x \in \{0,1\}^n} \left( \sum_{i=1}^{n} Q_{ii} x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} Q_{ij} x_i x_j \right)$$

* Historically, these models were solved by a few classical solvers.
* With technological advancements, an increasing number of quantum computers are supporting QUBOs as inputs.

* QUBO (Quadratic Unconstrained Binary Optimization) models are optimization models where constraints are relaxed. Here, no constraints, just an Objective Function based on binaries variables:

$$* \min_{x \in \{0,1\}^n} \left( \sum_{i=1}^{n} Q_{ii} x_i + \sum_{i=1}^{n} \sum_{j=i+1}^{n} Q_{ij} x_i x_j \right)$$

* Historically, these models were solved by a few classical solvers.
* With technological advancements, an increasing number of quantum computers are supporting QUBOs as inputs.
* It is likely that QUBO models will become the primary method for modeling mathematical programs in the future.

## >>> What is QUBO?

* Quadratic Unconstrained Binary Optimization (QUBO) is a type of mathematical programming problem that minimizes a quadratic function of binary variables.

## >>> What is QUBO?

* Quadratic Unconstrained Binary Optimization (QUBO) is a type of mathematical programming problem that minimizes a quadratic function of binary variables.
* All constraints should be relaxed into the objective function, and a penalty should be applied to the objective function if any relaxed constraint is not respected.

## >>> What is QUBO?

* Quadratic Unconstrained Binary Optimization (QUBO) is a type of mathematical programming problem that minimizes a quadratic function of binary variables.
* All constraints should be relaxed into the objective function, and a penalty should be applied to the objective function if any relaxed constraint is not respected.
* QUBO problems have a wide range of applications, including machine learning, combinatorial optimization, and finance. We can transform any linear mathematical programs into a QUBO.

>>> What is QUBO?

* Quadratic Unconstrained Binary Optimization (QUBO) is a type of mathematical programming problem that minimizes a quadratic function of binary variables.
* All constraints should be relaxed into the objective function, and a penalty should be applied to the objective function if any relaxed constraint is not respected.
* QUBO problems have a wide range of applications, including machine learning, combinatorial optimization, and finance. We can transform any linear mathematical programs into a QUBO.

Example

A first QUBO example:

$$\begin{aligned} \min \quad & x_1^2 + x_2^2 + 2x_1 + x_2 \\ \text{s.t.} \quad & x_1, x_2 \in \{0, 1\} \end{aligned}$$

## >>> QUBO Model with Matrix Representation

The general form of a QUBO problem can be expressed as:

$$\text{Minimize } f(x) = \sum_{i=1}^{n} \sum_{j=1}^{n} Q_{ij} x_i x_j$$

where:

* $x = \{x_1, x_2, \ldots, x_n\}$ are binary variables $(x_i \in \{0, 1\})$.
* $Q$ is an $n \times n$ matrix of coefficients.

The objective is to find the values of $x$ that minimize $f(x)$.

## >>> Example of a QUBO Model with Matrix Representation

Consider the QUBO expressed with this objective function:

$$f_{QUBO}(x) = 3x_1^2 - 2x_1x_2 + 4x_2^2$$

with:

$$x_1, x_2 \in \{0, 1\}$$

## >>> Example of a QUBO Model with Matrix Representation

Consider the QUBO expressed with this objective function:

$$f_{QUBO}(x) = 3x_1^2 - 2x_1x_2 + 4x_2^2$$

with:

$$x_1, x_2 \in \{0, 1\}$$

This can be represented as a QUBO matrix:

$$Q = \begin{bmatrix} 3 & -2 \\ 0 & 4 \end{bmatrix}$$

Here, the diagonal elements (3 and 4) correspond to the coefficients of $x_1^2$ and $x_2^2$, and the off-diagonal element represents the interaction between $x_1$ and $x_2$.

## >>> QUBO Solvers

Solvers for classical computers:

* GroverOptimizer
* Gurubi
* IBM CPLEX

## >>> QUBO Solvers

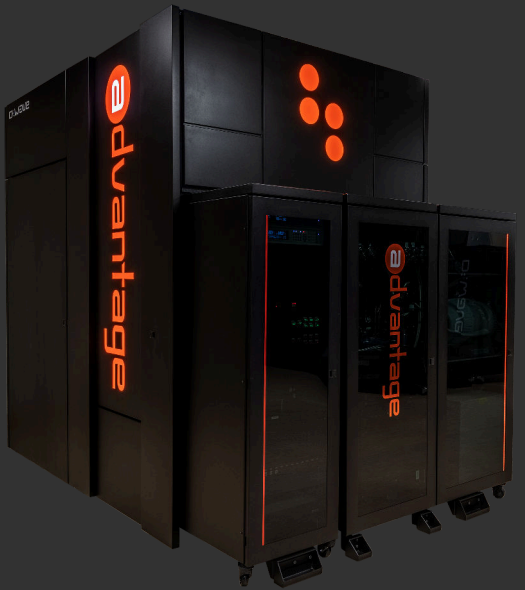
Solvers for classical computers:

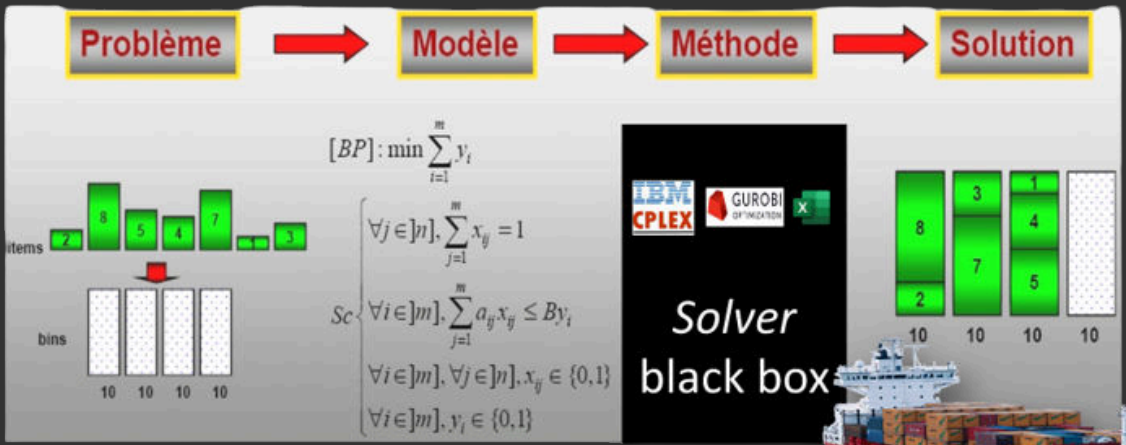
* GroverOptimizer
* Gurubi
* IBM CPLEX

Solver for Quantum machines:

* Pasqal
* D-Wave Systems Quantum Annealers
* Google Quantum Platform
* IBM Quantum Experience
* Rigetti Computing Forest

>>> D-Wave Analog Quantum Machines.  INPUT == QUBO




Problème
Modèle
Méthode
Solution
$[BP]: \min \sum_{i=1}^{m} y_i$
$Sc \begin{cases} \forall j \in ]n], \sum_{j=1}^{m} x_{ij} = 1 \\ \forall i \in ]m], \sum_{j=1}^{m} a_{ij} x_{ij} \le B y_i \\ \forall i \in ]m], \forall j \in ]n], x_{ij} \in \{0,1\} \\ \forall i \in ]m], y_i \in \{0,1\} \end{cases}$
items
bins
7
8
5
4
7
3
10 10 10 10
IBM CPLEX
GUROBI OPTIMIZATION
Solver
black box
8
2
3
7
1
4
5
10 10 10 10

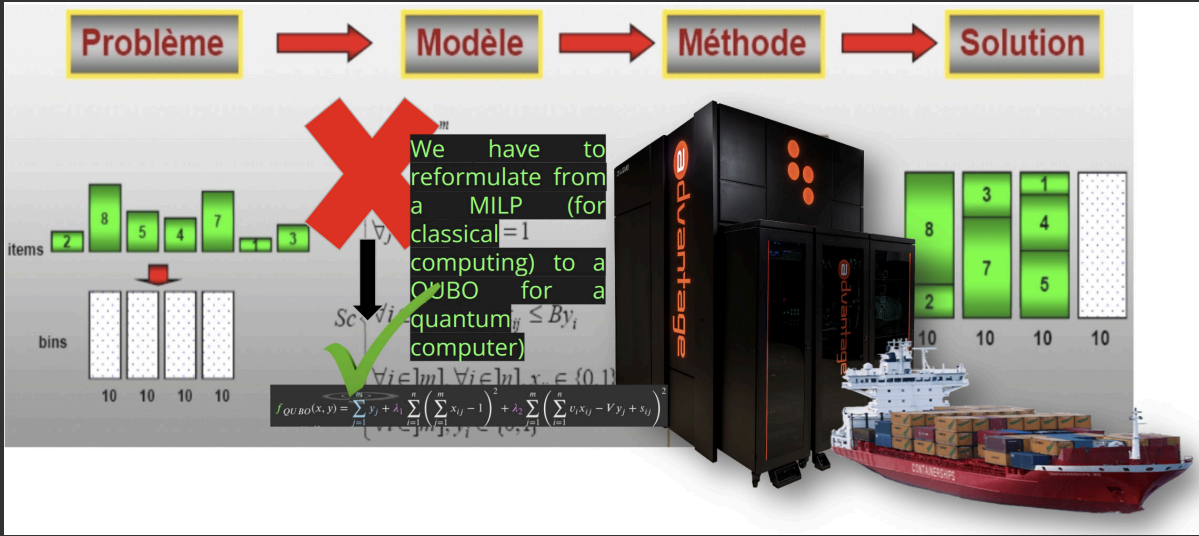



Figure 12: A QUBO of the Bin Packing Problem

## >>> Preparing the INPUT: from the MILP to the QUBO.

There are many Mixed Integer Linear Programs (MILP), QUBO models are much less common.

## >>> Preparing the INPUT: from the MILP to the QUBO.

There are many Mixed Integer Linear Programs (MILP), QUBO models are much less common.

Example: the Bin Packing problem reformulation

## >>> Preparing the INPUT: from the MILP to the QUBO.

There are many Mixed Integer Linear Programs (MILP), QUBO models are much less common.

Example: the Bin Packing problem reformulation

* $x$ is a binary vector such that for each $x_{ij}$, $i = 1..n, j = 1..m$, we have:

$$x_{ij} = \begin{cases} 1 & \text{if object } i \text{ is placed in bin } j, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

## >>> Preparing the INPUT: from the MILP to the QUBO.

There are many Mixed Integer Linear Programs (MILP), QUBO models are much less common.

Example: the Bin Packing problem reformulation

* $x$ is a binary vector such that for each $x_{ij}$, $i = 1..n, j = 1..m$, we have:

$$x_{ij} = \begin{cases} 1 & \text{if object } i \text{ is placed in bin } j, \\ 0 & \text{otherwise.} \end{cases} \tag{1}$$

* $y$ is a binary vector such that, for each bin $j$, where $j = 1..m$:

$$y_j = \begin{cases} 1 & \text{if bin } j \text{ is used,} \\ 0 & \text{otherwise.} \end{cases} \tag{2}$$

| Symbol | Description |
|---|---|
| $n$ | The total number of objects to be packed. |
| $m$ | The total number of bins. |
| $V$ | The capacity of each bin. |
| $v_i$ | The capacity consumption of object $i$. |

Table 2: Bin Packing Problem Parameters [Deleplanque et al., 2025a]

## >>> Mixed Integer Linear Problem (MILP) and QUBO

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i = 1, \ldots, n. \quad \Rightarrow \quad P1(x) = \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2 . \tag{3}$$

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i = 1, \dots, n. \quad \Rightarrow \quad P1(x) = \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2. \tag{3}$$

$$\sum_{i=1}^{n} v_i x_{ij} \leq V y_j \quad \forall j = 1, \dots, m. \quad \Rightarrow \quad P2(x, y) = \sum_{j=1}^{m} \left( \sum_{i=1}^{n} v_i x_{ij} - V y_j + s_{ij} \right)^2. \tag{4}$$

with $s_{ij} = \sum_{t=0}^{f(V)} 2^t z^t \quad \forall i = 1..n, j = 1..m.$

## >>> Mixed Integer Linear Problem (MILP) and QUBO

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i = 1, \ldots, n. \quad \Rightarrow \quad P1(x) = \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2. \tag{3}$$

$$\sum_{i=1}^{n} v_i x_{ij} \leq V y_j \quad \forall j = 1, \ldots, m. \quad \Rightarrow \quad P2(x, y) = \sum_{j=1}^{m} \left( \sum_{i=1}^{n} v_i x_{ij} - V y_j + s_{ij} \right)^2. \tag{4}$$

with $s_{ij} = \sum_{t=0}^{f(V)} 2^t z^t \quad \forall i = 1..n, j = 1..m.$

Objective of the $\mathcal{MILP}$ ⇓ the full $\mathcal{QUBO}$

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i = 1, \ldots, n. \quad \Rightarrow \quad P1(x) = \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2. \tag{3}$$

$$\sum_{i=1}^{n} v_i x_{ij} \leq V y_j \quad \forall j = 1, \ldots, m. \quad \Rightarrow \quad P2(x, y) = \sum_{j=1}^{m} \left( \sum_{i=1}^{n} v_i x_{ij} - V y_j + s_{ij} \right)^2. \tag{4}$$

with $s_{ij} = \sum_{t=0}^{f(V)} 2^t z^t \quad \forall i = 1..n, j = 1..m.$

Objective of the $\mathcal{MILP}$ ⇓ the full $\mathcal{QUBO}$

⇓ Minimize $f(x, y) = \sum_{j=1}^{m} y_j.$ ⇓ (5)

Constraints of the $\mathcal{MILP}$ => Penalties of the $\mathcal{QUBO}$

$$\sum_{j=1}^{m} x_{ij} = 1 \quad \forall i = 1, \ldots, n. \quad \Rightarrow \quad P1(x) = \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2. \tag{3}$$

$$\sum_{i=1}^{n} v_i x_{ij} \leq V y_j \quad \forall j = 1, \ldots, m. \quad \Rightarrow \quad P2(x,y) = \sum_{j=1}^{m} \left( \sum_{i=1}^{n} v_i x_{ij} - V y_j + s_{ij} \right)^2. \tag{4}$$

with $s_{ij} = \sum_{t=0}^{f(V)} 2^t z^t \quad \forall i = 1..n, j = 1..m.$

Objective of the $\mathcal{MILP}$ ⇓ the full $\mathcal{QUBO}$

⇓ Minimize $f(x,y) = \sum_{j=1}^{m} y_j.$ ⇓ (5)

Minimize $f_{QUBO}(x,y) = \sum_{j=1}^{m} y_j + \lambda_1 \sum_{i=1}^{n} \left( \sum_{j=1}^{m} x_{ij} - 1 \right)^2 + \lambda_2 \sum_{j=1}^{m} \left( \sum_{i=1}^{n} v_i x_{ij} - V y_j + s_{ij} \right)^2.$ (6)

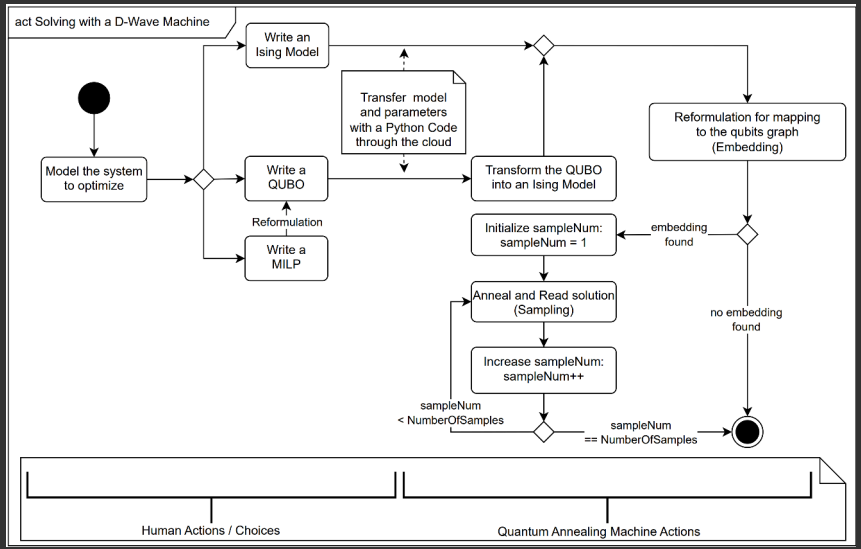


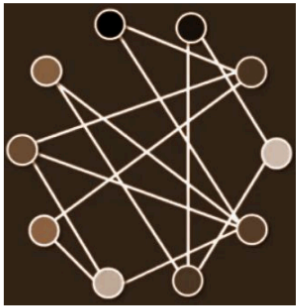

Figure 13: UML activity diagram: Human Vs Machines actions.

# Max-Cut Problem

## >>> Example of a Max-Cut problem: splitting guests, reducing chaos



* We need to split wedding guests into two tables.
* Some pairs of people strongly dislike each other.
* If they sit together, they may cause arguments or chaos.
* The goal is to separate as many “problematic pairs” as possible.
* This matches the Max-Cut problem: divide guests into two groups while maximizing the number of conflict edges cut.

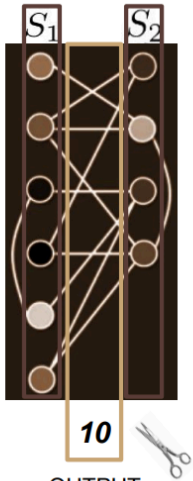


INPUT

## The Max-Cut Problem

**Finding 2 sets of nodes S1 and S2**, such that the **total cost** of the edges between the set S1 and the set S2 is **as large as possible**.

(here: no weighted)




OUTPUT

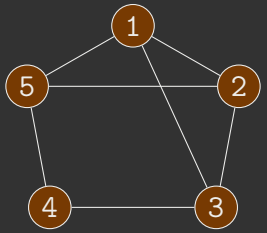
## >>> Ising model and the Max-Cut Problem

* We note $\mathcal{G}$ an undirected graph, $\mathcal{E}$ the set of edges, and $\mathcal{X}$ the set of vertices,
* We note $z$ is a $\{-1,1\}$-binary vector, such that $z_i = 1$ if the vertex $i$ is in one set, or $z_i = -1$ if the vertex $i$ is in the other set.

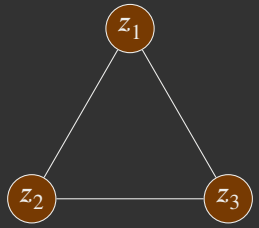
## >>> Ising model and the Max-Cut Problem

* We note $\mathcal{G}$ an undirected graph, $\mathcal{E}$ the set of edges, and $\mathcal{X}$ the set of vertices,
* We note $z$ is a $\{-1,1\}$-binary vector, such that $z_i = 1$ if the vertex $i$ is in one set, or $z_i = -1$ if the vertex $i$ is in the other set.
* An Ising model of the Max-Cut Problem can be expressed with the minimization of the following Objective function:

$$f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j \tag{7}$$



## >>> Exercise: The Frustrated Triangle ($f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j$)



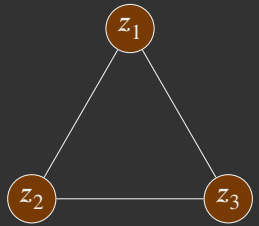

Consider a simple triangle graph ($K_3$).
We want to **minimize**:

$$f_{Ising}(z) = z_1 z_2 + z_2 z_3 + z_3 z_1$$

Questions:

1. Can we have $z_i z_j = -1$ for all 3 edges simultaneously?

## >>> Exercise: The Frustrated Triangle ($f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j$)



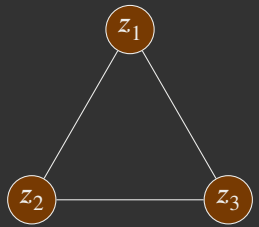

Consider a simple triangle graph ($K_3$).
We want to minimize:

$$f_{Ising}(z) = z_1 z_2 + z_2 z_3 + z_3 z_1$$

Questions:

1. Can we have $z_i z_j = -1$ for all 3 edges simultaneously?
2. What is the minimum possible value of $f_{Ising}$?

## >>> Exercise: The Frustrated Triangle ($f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j$)




Consider a simple triangle graph ($K_3$).
We want to **minimize**:

$$f_{Ising}(z) = z_1 z_2 + z_2 z_3 + z_3 z_1$$

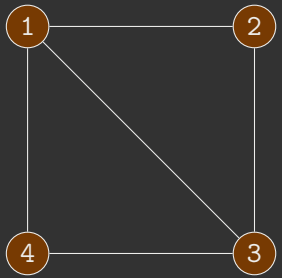
Questions:

1. Can we have $z_i z_j = -1$ for all 3 edges simultaneously?
2. What is the minimum possible value of $f_{Ising}$?

### Analysis

* It is impossible to have 3 unsatisfied edges (product of $z_i z_j$ around a loop must be $+1$).
* Best case: Two edges are $-1$ (cut), one is $+1$ (uncut).
* Minimum Energy: $(-1) + (-1) + (1) = \mathbf{-1}$.

## >>> Exercise: Calculation Example ($f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j$)

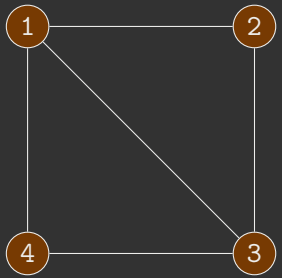



System: 5 edges, 4 vertices.

Task:

1. Propose a configuration $z$ that you think minimizes the energy.
2. Calculate $f_{Ising}(z)$ for this configuration.

## >>> Exercise: Calculation Example ($f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j$)




System: 5 edges, 4 vertices.

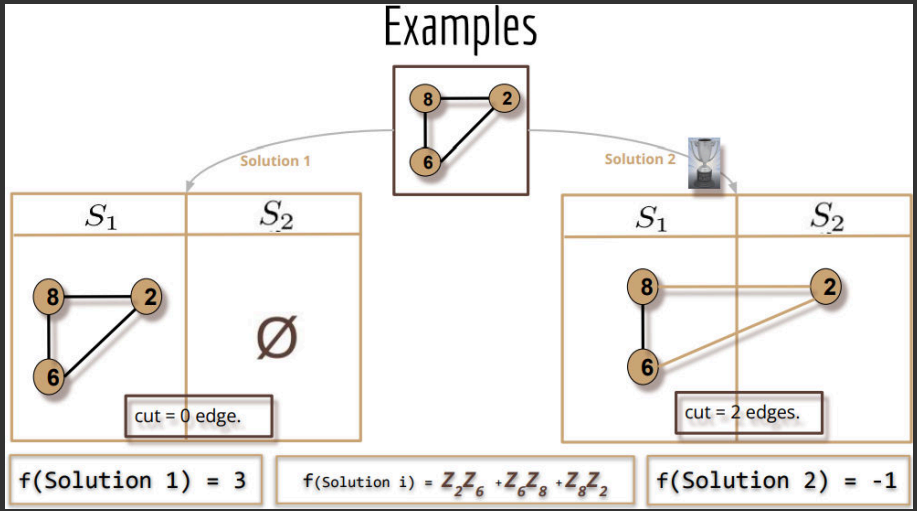
Task:

1. Propose a configuration $z$ that you think minimizes the energy.
2. Calculate $f_{Ising}(z)$ for this configuration.

### Solution Example

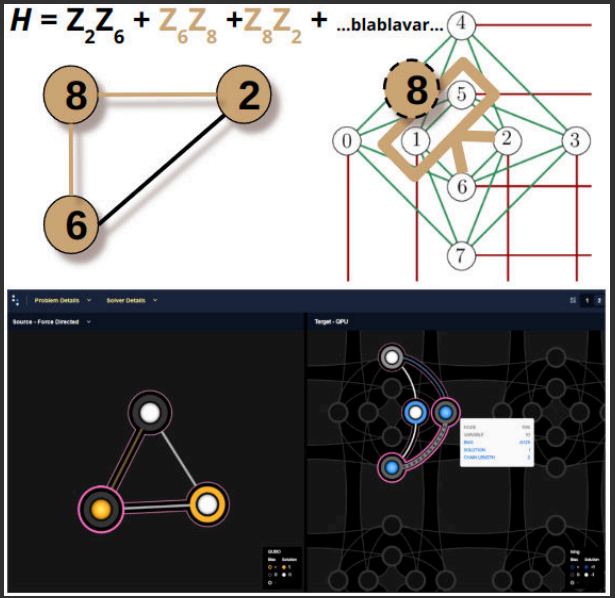
Try bipartite-like split: $z_1 = 1, z_3 = 1$ (same) and $z_2 = -1, z_4 = -1$ (same).

* Cut edges ($z_i z_j = -1$): (1,2), (2,3), (3,4), (4,1) $\rightarrow$ 4 edges.
* Uncut edges ($z_i z_j = 1$): (1,3) $\rightarrow$ 1 edge.
* $f_{Ising} = (-1)\times 4 + (1)\times 1 = \mathbf{-3}$.

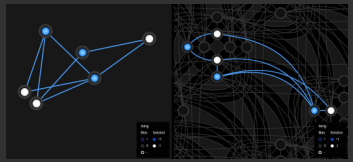



$$f_{Ising}(z) = \sum_{(i,j)\in\mathcal{E}} z_i z_j \tag{8}$$

# >>> Embedding process: the problem of low connectivity



>>> Solving a very small max-cut instance



TERMINAL

```
Le modele resolu est le suivant :
BinaryQuadraticModel({1: 0.0, 2: 0.0, 3: 0.0, 4: 0.0, 5: 0.0, 6: 0.0}, {(2, 1): 1.0, (3, 2): 1.0, (4, 1): 1.0, (4, 3): 1.0
, (5, 3): 1.0, (5, 4): 1.0, (6, 4): 1.0, (6, 5): 1.0}, 0.0, 'SPIN')
The solution obtained by D-Wave's quantum annealer Advantage_system4.1 is
   1  2  3  4  5  6 energy num_oc. chain_.
0 +1 -1 +1 -1 -1 +1   -6.0    3733     0.0
1 -1 +1 -1 +1 +1 -1   -6.0    4257     0.0
2 +1 -1 +1 -1 +1 -1   -4.0       1     0.0
3 -1 +1 -1 +1 -1 -1   -4.0       3     0.0
4 -1 +1 -1 +1 -1 +1   -4.0       1     0.0
5 -1 +1 +1 +1 -1 +1   -2.0       1     0.0
6 -1 +1 -1 +1 +1 +1   -2.0       1     0.0
7 +1 -1 -1 -1 +1 -1   -2.0       1     0.0
8 -1 -1 +1 +1 -1 -1   -2.0       1     0.0
9 -1 -1 +1 -1 -1 +1   -2.0       1     0.0
['SPIN', 10 rows, 8000 samples, 6 variables]
Leap IDE /workspace/DWAVEQuantum $
```

Figure 15: Results. Here: 8000 Anneals is done. 2 opt. solutions (-6) are found.

| | |
|---|---|
| $\|\mathcal{E}\|$ | Number of edges in the set E |
| $\|\mathcal{X}\|$ | Number of vertices in the set X |
| $\|Q\|$ | Number of qubits used by the embedding |
| $QPU_1$ | Time in microseconds for one anneal |
| $QPU_{10^4}$ | Time in microseconds for the 10000 anneals |
| $Cut_1$ | Cut value of the first solution obtained |
| $Cut_{10^4}$ | Cut value of the best solution obtained |
| $Cut_{Lit.}$ | Cut value of the literature best solution |

Table 3: XP parameters.

| inst | $\|\mathcal{E}\|$ | $\|\mathcal{X}\|$ | $\|Q\|$ | $QPU_1$ | $QPU_{10^4}$ | $Cut_1$ | $Cut_{10^4}$ | $Cut_{Lit.}$ |
|---|---|---|---|---|---|---|---|---|
| 51000 | 375 | 125 | 428 | 163.46$\mu$s | 1.804s | 102 | 110 | 110 |
| 52000 | 375 | 125 | 457 | 129.26$\mu$s | 1.017s | 106 | 112 | 112 |
| 53000 | 375 | 125 | 440 | 134.84$\mu$s | 1.822s | 92 | 106 | 106 |
| 54000 | 375 | 125 | 467 | 171.94$\mu$s | 1.930s | 100 | 114 | 114 |
| 55000 | 375 | 125 | 466 | 173.90$\mu$s | 1.030s | 96 | 112 | 112 |
| 56000 | 375 | 125 | 420 | 103.80$\mu$s | 1.763s | 98 | 110 | 110 |
| 57000 | 375 | 125 | 442 | 156.38$\mu$s | 1.367s | 94 | 112 | 112 |
| 58000 | 375 | 125 | 470 | 149.08$\mu$s | 1.799s | 98 | 108 | 108 |
| 59000 | 375 | 125 | 422 | 188.46$\mu$s | 1.227s | 92 | 110 | 110 |

Table 4: Results on [Festa et al., 2002] instances.

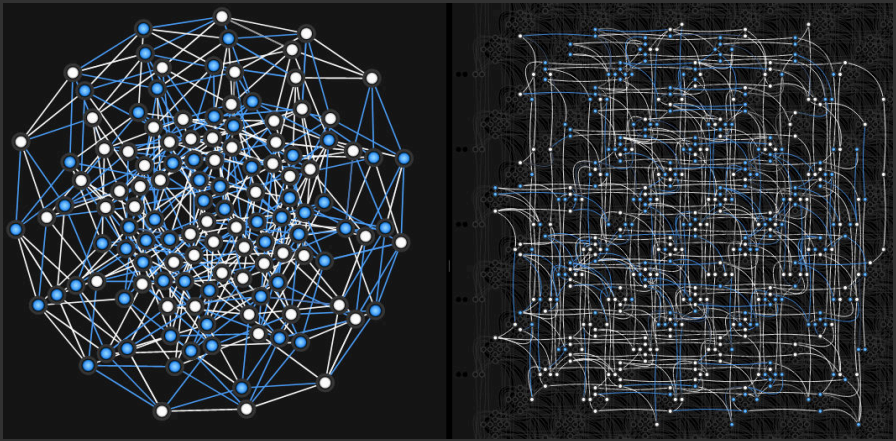
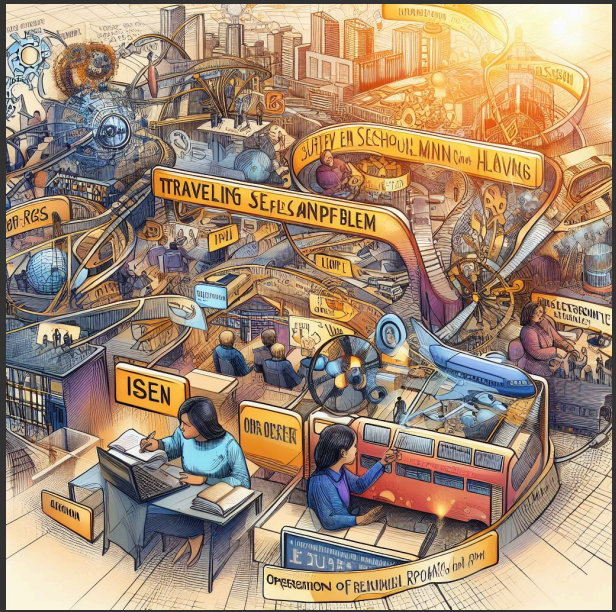


Figure 16: Graph representing the sg3dl051000 Max-Cut instance from [Festa et al., 2002] (left) and the corresponding qubits graph (right). Here the instance has 125 nodes and the machine requires 436 qubits. The two graphs has been drawn with the Inspector package of the Ocean D-Wave API.

# >>> Traveling Salesman Problem - Introduction

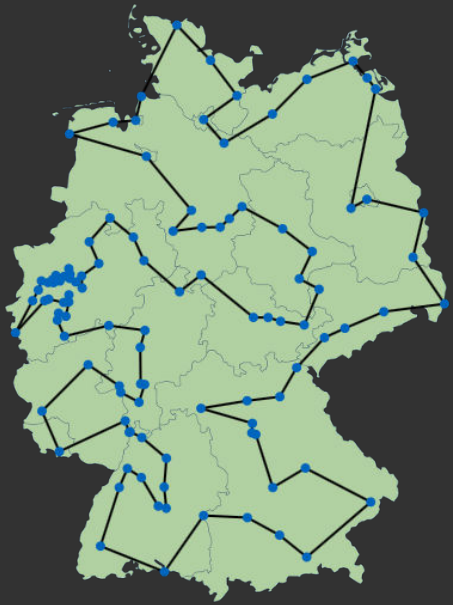


# Traveling Salesman Problem (TSP)

–

a NP-Hard problem

# >>> Traveling Salesman Problem - Introduction





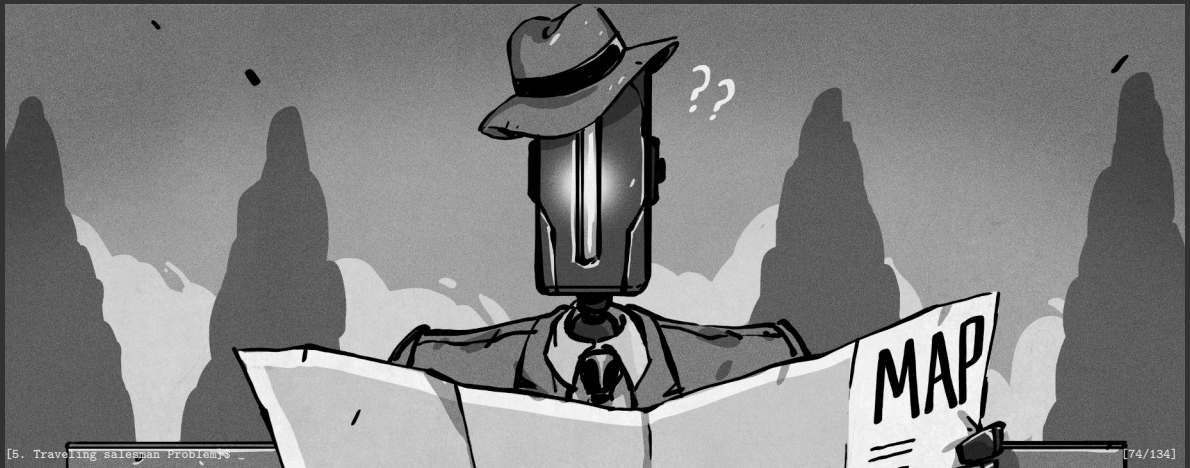

Let’s understand the model through practice!
(by starting with the 4 cities problem)

(a)

# QUBO
of the
# Traveling Salesman
Problem

>>> the Traveling Salesman Problem (TSP)

* The goal of the problem is to find the shortest possible route that a salesman can take to visit a set of cities and return to the starting point, visiting each city only once.
* The TSP is an NP-hard problem.



| | |
|---|---|
| $\mathcal{N}$ | The set of cities. |
| $\mathcal{P}$ | Consecutive cities (positions) in the route. |

Table 5: TSP problem parameters.

| | |
|---|---|
| $\mathcal{N}$ | The set of cities. |
| $\mathcal{P}$ | Consecutive cities (positions) in the route. |

Table 5: TSP problem parameters.

* We note $x$ a decision variable vector where each element $x_{ip}, i \in \mathcal{N}, p \in \mathcal{P}$, takes a $\{0;1\}$ value such that:

$$x_{ip} = \begin{cases} 1 & \text{if the city } i, i \in \mathcal{N}, \text{is visited at the } p^{th} \text{ position,} \\ 0 & \text{otherwise.} \end{cases}$$

## >>> TSP: a quadratic Objective function

* We avoid subtour constraints, the use of quadratic terms allow us to consider two consecutive customers in a the vehicle route by the product $x_{ip}x_{jp+1}$.

## >>> TSP: a quadratic Objective function

* We avoid subtour constraints, the use of quadratic terms allow us to consider two consecutive customers in a the vehicle route by the product $x_{ip}x_{jp+1}$.
* If both $i$ and $j$ are managed in succession, we pay the cost $c_{ij}$.

## >>> TSP: a quadratic Objective function

* We avoid subtour constraints, the use of quadratic terms allow us to consider two consecutive customers in a the vehicle route by the product $x_{ip}x_{jp+1}$.
* If both $i$ and $j$ are managed in succession, we pay the cost $c_{ij}$.
* The expression $c_{ij}x_{ip}x_{j1}$ considers the cost of the way back to the first element $(p = 1)$ from the last one $p = \mathcal{P}$.

* We avoid subtour constraints, the use of quadratic terms allow us to consider two consecutive customers in a the vehicle route by the product $x_{ip}x_{jp+1}$.
* If both $i$ and $j$ are managed in succession, we pay the cost $c_{ij}$.
* The expression $c_{ij}x_{ip}x_{j1}$ considers the cost of the way back to the first element $(p = 1)$ from the last one $p = \mathcal{P}$.
* We note $f_{TSP}$ the quadratic Objective function of the TSP given in expression (42).

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij}x_{i\mathcal{P}}x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij}x_{ip}x_{jp+1}) \quad (9)$$

* Constraints (10) force to satisfy all its $|\mathcal{N}|$ customers exactly once.

$$\sum_{p=1}^{|\mathcal{P}|} x_{ip} = 1 \qquad \forall i \in \mathcal{N} \tag{10}$$

* Constraints (10) force to satisfy all its $|\mathcal{N}|$ customers exactly once.

$$\sum_{p=1}^{|\mathcal{P}|} x_{ip} = 1 \qquad \forall i \in \mathcal{N} \tag{10}$$

* Constraints (11) force to satisfy exactly one customer at each position of its route.

$$\sum_{i=1}^{|\mathcal{N}|} x_{ip} = 1 \qquad \forall p \in \mathcal{P} \tag{11}$$

* Remark: $|\mathcal{N}| = |\mathcal{P}|$

## Mathematical Tools

* Relaxing an equality constraint...(tips: $x^2 = x, xbinary$)

$$((\sum_{i}^{n} x_i) - C)^2 = \sum_{i}^{n} x_i^2 + 2 \sum_{i}^{n} \sum_{j>i}^{n} x_i x_j - 2C \sum_{i}^{n} x_i + C^2 \quad (12)$$

* Since $x$ is a boolean vector, we obtain:

$$((\sum_{i}^{n} x_i) - C)^2 = -C \sum_{i}^{n} x_i + 2 \sum_{i}^{n} \sum_{j>i}^{n} x_i x_j + C^2 \quad (13)$$

>>> First set of constraints relaxation

Fist set of constraints:

$$\sum_{p=1}^{|\mathcal{P}|} x_{ip} = 1 \qquad \forall i \in \mathcal{N} \tag{14}$$

Using the following equality:

$$((\sum_{i}^{n} x_i) - C)^2 = -C \sum_{i}^{n} x_i + 2 \sum_{i}^{n} \sum_{j>i}^{n} x_i x_j + C^2 \tag{15}$$

Removing the constants, we finally obtain the following relaxed constraints:

$$Cons_1(x) = \lambda_1 \sum_{i}^{|\mathcal{N}|} (\sum_{p=1}^{|\mathcal{P}|} x_{ip} - 1)^2 = \lambda_1 \sum_{i}^{|\mathcal{N}|} (- \sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2 \sum_{p=1}^{|\mathcal{P}|} \sum_{q>p}^{|\mathcal{P}|} x_{ip} x_{iq}) \tag{16}$$

>>> Second set of constraints relaxation

Second set of constraints:

$$\sum_{i=1}^{|\mathcal{N}|} x_{ip} = 1 \qquad \forall p \in \mathcal{P} \tag{17}$$

Using the following equality:

$$((\sum_{i}^{n} x_i) - C)^2 = -C \sum_{i}^{n} x_i + 2 \sum_{i}^{n} \sum_{j>i}^{n} x_i x_j + C^2 \tag{18}$$

And removing the constants, we finally obtain the following relaxed constraints:

$$Cons_2(x) = \lambda_2 \sum_{p=1}^{|\mathcal{P}|} (\sum_{i=1}^{|\mathcal{N}|} x_{ip} - 1)^2 = \lambda_2 \sum_{p=1}^{|\mathcal{P}|} (- \sum_{i=1}^{|\mathcal{N}|} x_{ip} + 2 \sum_{i=1}^{|\mathcal{N}|} \sum_{j>i}^{|\mathcal{N}|} x_{ip} x_{jp}) \tag{19}$$

For a right couple $(\lambda_1, \lambda_2)$, we try to find $x$ minimizing $f_{TSP}^{QUBO}(x)$:

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{20}$$

For a right couple $(\lambda_1, \lambda_2)$, we try to find $x$ minimizing $f_{TSP}^{QUBO}(x)$:

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{20}$$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{i\mathcal{P}} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{21}$$

For a right couple $(\lambda_1, \lambda_2)$, we try to find $x$ minimizing $f_{TSP}^{QUBO}(x)$:

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{20}$$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{i\mathcal{P}} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{21}$$

$$Cons_1(x) = \lambda_1 \sum_{i=1}^{|\mathcal{N}|} (- \sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2 \sum_{p=1}^{|\mathcal{P}|} \sum_{q>p}^{|\mathcal{P}|} x_{ip} x_{iq}) \tag{22}$$

For a right couple $(\lambda_1, \lambda_2)$, we try to find $x$ minimizing $f_{TSP}^{QUBO}(x)$:

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{20}$$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{i\mathcal{P}} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{21}$$

$$Cons_1(x) = \lambda_1 \sum_{i=1}^{|\mathcal{N}|} (- \sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2 \sum_{p=1}^{|\mathcal{P}|} \sum_{q>p}^{|\mathcal{P}|} x_{ip} x_{iq}) \tag{22}$$

$$Cons_2(x) = \lambda_2 \sum_{p=1}^{|\mathcal{P}|} (- \sum_{i=1}^{|\mathcal{N}|} x_{ip} + 2 \sum_{i=1}^{|\mathcal{N}|} \sum_{j>i} x_{ip} x_{jp}) \tag{23}$$

>>> Exercise: Write $f_{TSP}^{QUBO}(x)$ such that $f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x)$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{ip} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{24}$$

$$Cons_1(x) = \lambda_1 \sum_{i=1}^{|\mathcal{N}|} (- \sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2 \sum_{p=1}^{|\mathcal{P}|} \sum_{q>p}^{|\mathcal{P}|} x_{ip} x_{iq}) \tag{25}$$

$$Cons_2(x) = \lambda_2 \sum_{p=1}^{|\mathcal{P}|} (- \sum_{i=1}^{|\mathcal{N}|} x_{ip} + 2 \sum_{i=1}^{|\mathcal{N}|} \sum_{j>i}^{|\mathcal{N}|} x_{ip} x_{jp}) \tag{26}$$

| | A | B | C | D | E |
|---|---|---|---|---|---|
| A | - | 5 | 10 | 3 | 10 |
| B | 5 | - | 7 | 5 | 8 |
| C | 10 | 7 | - | 6 | 1 |
| D | 3 | 5 | 6 | - | 4 |
| E | 10 | 8 | 1 | 4 | - |

# Practice:
# The Traveling Salesman Problem

on a quantum machine

# >>> Instances: TSP Lib Format

```
1
2  NAME: data/tsp_5.tsp
3  TYPE: TSP
4  DIMENSION: 5
5  EDGE_WEIGH_TYPE: "EXPLICIT"
6  EDGE_WEIGHT_FORMAT: "FULL_MATRIX"
7
8
9  EDGE_WEIGHT_SECTION
10 0    78    41    61    20
11 78     0    53    57    36
12 41    53     0    32    41
13 61    57    32     0    96
14 20    36    41    96     0
```

## >>> Instances: Generation (or not)

* You can take few instance files from the online folder.
* You can also generate instances with the code *tsp_file_generation.py* also given in the online folder.

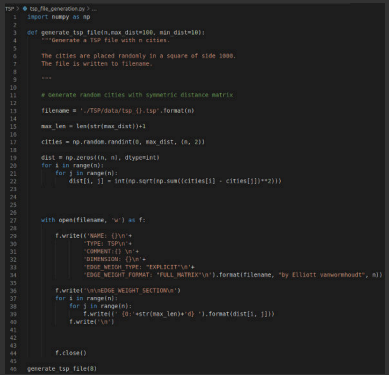



Figure 18: Code for generating a TSP Lib Format instance

# >>> Implementation: load an instance

```
1  def load_tsp(filename: str):
2  """     Load a TSP file and return the distance matrix.
3  The file must be in TSP LIB format.
4  """
5  with open(filename, 'r') as f:
6  lines = f.readlines()
7  start,n = 0,0
8  for i, line in enumerate(lines):
9  if line.startswith('DIMENSION'):
10 n = int(line.split(':')[-1])
11 if line.startswith('EDGE_WEIGHT_SECTION'):
12 start = i + 1
13
14 end = start + n
15
16 dist = np.zeros((n, n), dtype=int)
17 for i, line in enumerate(lines[start:end]):
18 dist[i, :] = np.array([int(x) for x in line.split()])
19 return dist
```

## >>> Declare a bqm model in Python

```
1  # Constants
2  NUM_CITY = 8
3  TOPOLOGY = "pegasus"
4  LAMBDA = 125
5  NUM_READS = 1000
6
7  # We load the instance
8  matrix = load_tsp('./TSP/data/tsp_'+str(NUM_CITY)+'.tsp')
9
10 # We set the model and the quantum machine
11
12 sampler =
13 AutoEmbeddingComposite(DWaveSampler(solver={'topology__type': TOPOLOGY}))
14
15 # We create the Binary Quadratic Model (bqm) according to the matrix
16 bqm = create_bqm(matrix)
```

* *create_bqm*(*matrix*) will create the binary quadratic model to solve as it is shown in the following slides.

* $n$ is the number of cities
* $bqm$ is our model

```
1 n = len(matrix)
2 bqm = BinaryQuadraticModel.empty(dimod.BINARY)
```

* We now code the QUBO $f_{TSP}^{QUBO}(x)$:

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{27}$$

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{28}$$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{ip} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{29}$$

```
for i in range(n):
    for j in range(n):
        if i != j:
            for p in range(n):
                bqm.add_quadratic('x_{}_{}'.format(i, p), 'x_{}_{}'.format(j, (p+1)%n), matrix[i, j])
```

Figure 19: Code of the TSP quadratic objective function $f_{TSP}(x)$

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{30}$$

$$Cons_1(x) = \lambda_1 \sum_{i=1}^{|\mathcal{N}|}(-\sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2\sum_{p=1}^{|\mathcal{P}|}\sum_{q>p}^{|\mathcal{P}|} x_{ip}x_{iq}) \tag{31}$$

```
for i in range(n):
    for p in range(n):
        for q in range(p+1, n):
            bqm.add_quadratic('x_{}_{}'.format(i, p), 'x_{}_{}'.format(i, q), 2*LAMBDA)
        bqm.add_linear('x_{}_{}'.format(i, p), -LAMBDA)
```

Figure 20: Code of the first set of constraints $Cons_1(x)$ to relax

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{32}$$

$$Cons_2(x) = \lambda_2 \sum_{p=1}^{|\mathcal{P}|}(- \sum_{i=1}^{|\mathcal{N}|} x_{ip} + 2 \sum_{i=1}^{|\mathcal{N}|} \sum_{j>i} x_{ip}x_{jp}) \tag{33}$$

```
for p in range(n):
    for i in range(n):
        for j in range(i+1, n):
            bqm.add_quadratic('x_{}_{}'.format(i, p), 'x_{}_{}'.format(j, p), 2*LAMBDA)
        bqm.add_linear('x_{}_{}'.format(i, p), -LAMBDA)
```

Figure 21: Code of the second set of constraints $Cons_2(x)$ to relax

* Do not forget: *return the bqm*

## >>> Constants part into the offset

* Adding the constants allows to have the energy of the solution matching the initial Objective function of the QUBO (even if that will not have an impact on the exploration of the solution space).

## >>> Constants part into the offset

* Adding the constants allows to have the energy of the solution matching the initial Objective function of the QUBO (even if that will not have an impact on the exploration of the solution space).
* For each set of constraints, we have:
  * for i in range(n):
    * bqm.offset += LAMBDA
  * for p in range(n):
    * bqm.offset += LAMBDA
  * Of course you can simplify that in one loop or integrate the offset while relaxed constraints enter the bqm.

## >>> Solving the QUBO

We set:

* the number of reads $num_reads$,
* the annealing time $annealing_time$,

and we launch the resolution process:

```
1 sampleset =
2 sampler.sample(bqm,num_reads=500,annealing_time=0.5)
```

## >>> Solving the QUBO

We set:

* the number of reads *num_reads*,
* the annealing time *annealing_time*,

and we launch the resolution process:

```
sampleset =
sampler.sample(bqm,num_reads=500,annealing_time=0.5)
```

We display the data from the embedding process such that:

* the number of variables,
* the number of physical qubits,

with the following code:

```
embedding = sampleset.info['embedding_context']['embedding']
print(f"Number of logical variables: {len(embedding.keys())}")
print(f"Number of physical qubits used in embedding:
{sum(len(chain) for chain in embedding.values())}")
```

>>> First Results

| | x_0_0 | x_0_1 | x_0_2 | x_0_3 | x_0_4 | x_1_0 | x_1_1 | ... | x_4_4 | energy | num_oc. | ... |
|---|---|---|---|---|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 1 | 0 | 0 | ... | 0 | 186.0 | 2 | ... |
| 1 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 1 | 186.0 | 1 | ... |
| 2 | 0 | 0 | 0 | 1 | 0 | 1 | 0 | ... | 1 | 186.0 | 1 | ... |
| 15 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | ... | 0 | 186.0 | 1 | ... |
| 3 | 0 | 0 | 0 | 1 | 0 | 0 | 1 | ... | 0 | 202.0 | 1 | ... |
| 21 | 0 | 0 | 0 | 0 | 1 | 0 | 1 | ... | 0 | 202.0 | 1 | ... |
| 4 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 232.0 | 1 | ... |
| 5 | 1 | 0 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 232.0 | 1 | ... |
| 102 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 232.0 | 1 | ... |
| 260 | 0 | 1 | 0 | 0 | 0 | 0 | 0 | ... | 0 | 236.0 | 1 | ... |
| 6 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | ... | 0 | 248.0 | 1 | ... |
| 7 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | ... | 0 | 279.0 | 1 | ... |
| 8 | 0 | 0 | 1 | 0 | 0 | 0 | 0 | ... | 1 | 283.0 | 1 | ... |
| 25 | 0 | 0 | 0 | 1 | 0 | 0 | 0 | ... | 0 | 283.0 | 1 | ... |
| 9 | 0 | 0 | 0 | 0 | 1 | 1 | 0 | ... | 0 | 313.0 | 1 | ... |

Figure 22: Results with optimal sol. Obj. value equal to 186. Command: print(sampleset)

## >>> First Results

* We can show the number of variables of the problem and the number of qubits used with:
  * embedding = sampleset.info['embedding_context']['embedding']
  * print(f"Number of logical variables:  len(embedding.keys())")
  * print(f"Number of physical qubits used in embedding:  sum(len(chain) for chain in embedding.values())")

```
Number of logical variables: 25
Number of physical qubits used in embedding: 77
```

Figure 23: Embedding process characteristics. The initial TSP instance has 5 cities, and since the QUBO considers a complete graph, the number of variables is $|\mathcal{N}|^2 = 25$. The embedding process needs 77 qubits which is more than three times the number of original variables.

* You can check the feasability of the solution with the help of the verify_solution function:

```
1 def verify_solution(solution, matrix):
2
3 solution = [k for k, v in solution.items() if v == 1]
4 solution = [(int(x.split('_')[1]),int(x.split('_')[2])) for x in solution]
5 if len(solution) != len(matrix):
6 return False
7 if list(range(len(matrix))) !=list(sorted([x[0] for x in solution])):07287029
8 return False
9 return True
```

## >>> Output analysis

* You can display a solution with the help of the pretty_print function:

```
1 def pretty_print(solution, matrix):
2
3 solution = [k for k, v in solution.items() if v == 1]
4 solution = [(int(x.split('_')[1]),int(x.split('_')[2])) for x in solution]
5 solution_sorted = list(sorted(solution, key=lambda x: x[1]))
6 solution_sorted = [x[0] for x in solution_sorted]
7 print(solution_sorted)
8 print(cost_function(solution_sorted, matrix))
9 return solution_sorted
10
11 def cost_function(x, matrix):
12 return np.sum(np.fromiter((matrix[x[i], x[i + 1]] for i in range(len(x) - 1)), dtype=
      float)) + matrix[x[-1], x[0]]
```

* You can display feasible solutions and focus on the best one with the following print_sampleset function:

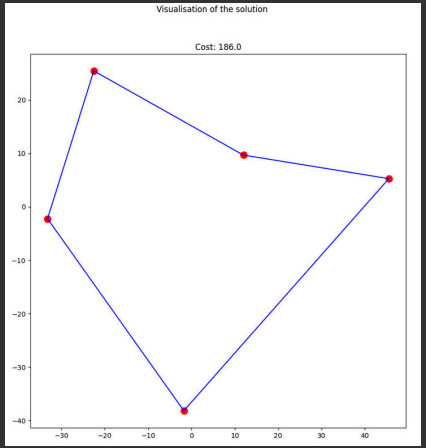
```
1  def print_sampleset(matrix,sampleset):
2
3  verified_samples = []
4  for i, sample in enumerate(sampleset):
5  if verify_solution(sample, matrix):
6  solution = pretty_print(sample,matrix)
7  verified_samples.append(solution)
8
9  costs = [cost_function(x,matrix) for x in verified_samples]
10 if len(verified_samples) == 0:
11 print("No solution found")
12 return
13 print("Best solution: ", verified_samples[np.argmin(costs)])
14 print("Cost: ", np.min(costs))
```

# >>> Output analysis

```
[3, 2, 0, 4, 1]
186.0
[3, 2, 4, 0, 1]
228.0
[2, 3, 1, 0, 4]
228.0
[1, 2, 4, 0, 3]
232.0
[1, 0, 3, 2, 4]
248.0
[3, 2, 4, 1, 0]
248.0
[1, 2, 0, 4, 3]
267.0
[0, 2, 3, 4, 1]
283.0
[2, 3, 4, 1, 0]
283.0
[1, 2, 0, 3, 4]
287.0
[0, 2, 4, 3, 1]
313.0
[1, 2, 4, 3, 0]
329.0
[2, 3, 1, 4, 0]
186.0
[2, 4, 1, 3, 0]
236.0
[2, 3, 0, 1, 4]
248.0
[2, 1, 0, 4, 3]
279.0
[3, 4, 0, 1, 2]
279.0
Best solution:  [3, 2, 0, 4, 1]
Cost:  186.0
```

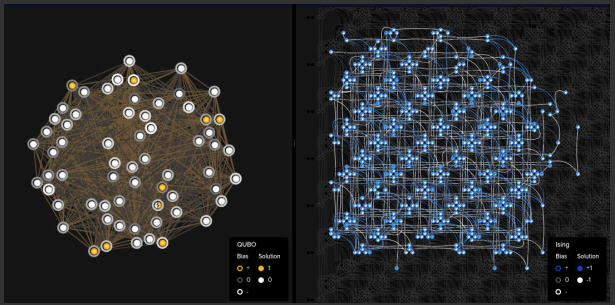
## >>> Drawing a solution; example of optimal solution with 5 cities

* You can also draw you solution with the "plot solution" function given in the online drive.



## >>> Display the TSP instance graph and its embedding into the topology

* You just have to use the inspector object:
  * import dwave.inspector as inspector
  * # [...]
  * inspector.show(sampleset)



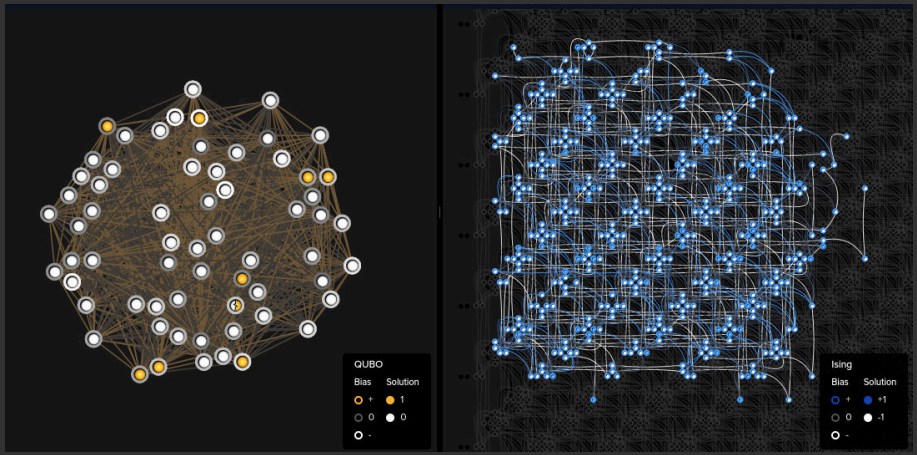



Figure 25: Instance graph and its mapping into the Pegasus topology: from 8 cities and its 64 variables to 455 qubits.

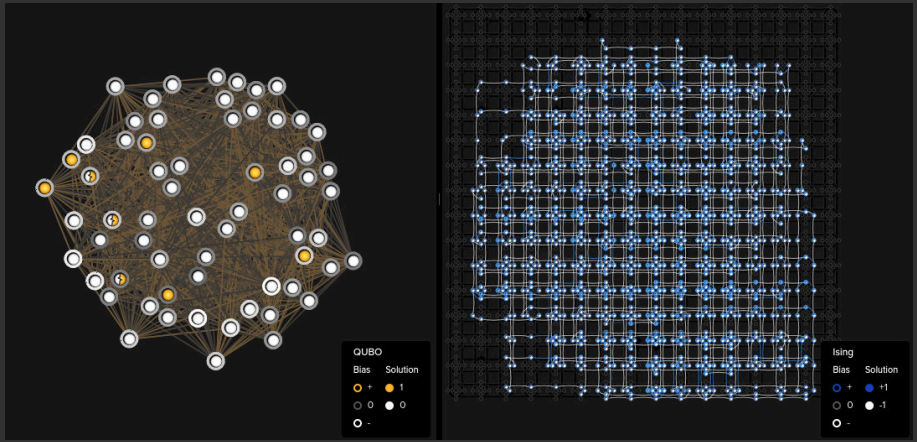



Figure 26: Instance graph and its mapping into the Chimera topology: from 8 cities and its 64 variables to 1103 qubits.

# Job-Shop Problem

–

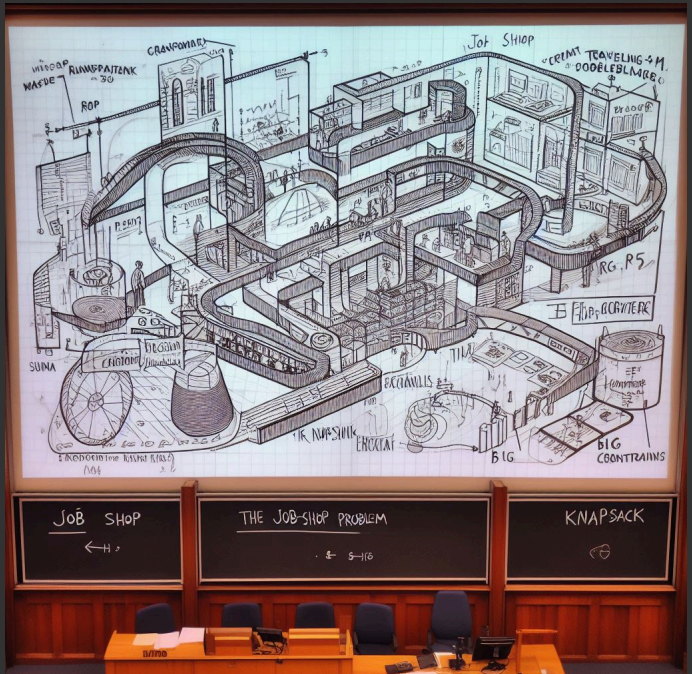
a work with

–

Riad Aggoune
(List, Luxembourg)

–

[Aggoune and Deleplanque, 2023]


JOB SHOP
THE JOB-SHOP PROBLEM
KNAPSACK

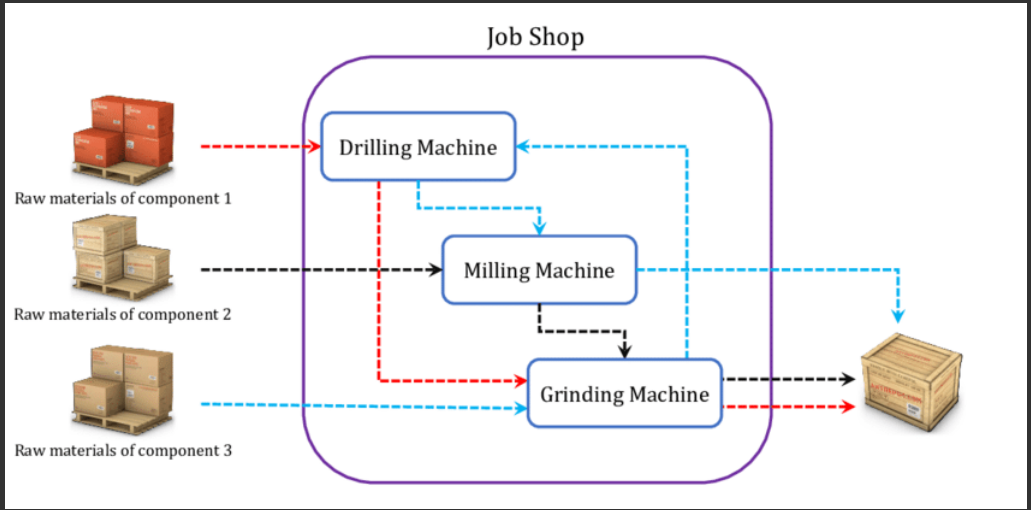



Figure 27: Example with 3 machines and 3 jobs of 2 or 3 operations. Source: [Yazdani et al., 2017]

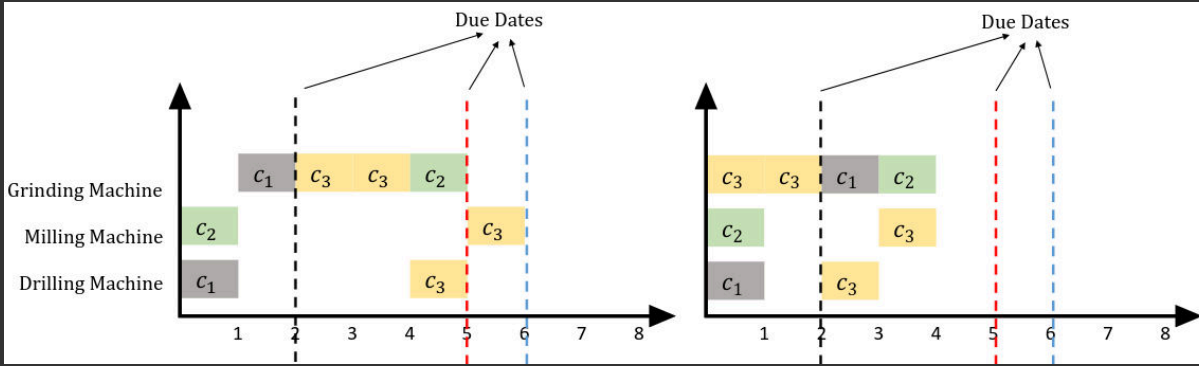



Figure 28: A bad (left) and a good (right) solutions. The former takes time while the latter minimizes the makespan. Source: [Yazdani et al., 2017]

## >>> Introduction to the Job-Shop Problem

The job shop problem in few words:

* We want to schedule a set of jobs on a set of machines minimizing the makespan.

## >>> Introduction to the Job-Shop Problem

The job shop problem in few words:

* We want to schedule a set of jobs on a set of machines minimizing the makespan.
* Each job consists of a sequence of operations, which must be processed in a specific order.

## >>> Introduction to the Job-Shop Problem

The job shop problem in few words:

* We want to schedule a set of jobs on a set of machines minimizing the makespan.
* Each job consists of a sequence of operations, which must be processed in a specific order.
* Each operation requires a specific machine for a specific duration.

## >>> Introduction to the Job-Shop Problem

The job shop problem in few words:

* We want to schedule a set of jobs on a set of machines minimizing the makespan.
* Each job consists of a sequence of operations, which must be processed in a specific order.
* Each operation requires a specific machine for a specific duration.
* This is a problem similar to the RCPSP.

# >>> MILP of the Job-Shop Problem with big-M

$J$ = set of jobs, $MC$ = set of machines,

$O_j$ = ordered set of operations for job $j$

$p_{jo}$ = processing time of job $j$ for operation $o$

$s_{jo}$ = start time of job $j$ for operation $o$

$C_{max}$ = makespan, i.e., maximum completion time

$y_{jo,j'o'}$ = binary variable: 1 if operation $jo$ precedes operation $j'o'$

## >>> MILP of the Job-Shop Problem with big-M

$J$ = set of jobs, $MC$ = set of machines,

$O_j$ = ordered set of operations for job $j$

$p_{jo}$ = processing time of job $j$ for operation $o$

$s_{jo}$ = start time of job $j$ for operation $o$

$C_{max}$ = makespan, i.e., maximum completion time

$y_{jo,j'o'}$ = binary variable: 1 if operation $jo$ precedes operation $j'o'$

Minimize $C_{max}$

$$s_{j,o} + p_{jo} \leq s_{j,o+1} \quad \forall j \in J, o \in O_j - 1$$

$$s_{j,o} + p_{jo} \leq C_{max} \quad \forall j \in J, o \in O_j$$

$$s_{j,o} + p_{jo} \leq s_{j',o'} + M(1 - y_{jo,j'o'}) \quad \forall \text{ pairs } (jo, j'o') \text{ sharing mach.}$$

$$s_{j',o'} + p_{j'o'} \leq s_{j,o} + M y_{jo,j'o'} \quad \forall \text{ pairs } (jo, j'o') \text{ sharing mach.}$$

>>> JSSP QUBO ([Rieffel et al., 2015] [Carugno et al., 2022])

* The boolean variable $x_{ij}^{t}$ takes the value 1 if the operation $i$ of the job $j$ starts in period $t$, with $i = 1..n_i$, $j = 1..n$, $t = 1..T$, and takes the value 0 otherwise.

* The boolean variable $x_{ij}^{t}$ takes the value 1 if the operation $i$ of the job $j$ starts in period $t$, with $i = 1..n_i$, $j = 1..n$, $t = 1..T$, and takes the value 0 otherwise.
* We note $M_{ij}$, $i = 1..n_i, j = 1..n$, the required machine for the operation $i$ of the job $j$.

* The boolean variable $x_{ij}^{t}$ takes the value 1 if the operation $i$ of the job $j$ starts in period $t$, with $i = 1..n_i$, $j = 1..n$, $t = 1..T$, and takes the value 0 otherwise.
* We note $M_{ij}$, $i = 1..n_i, j = 1..n$, the required machine for the operation $i$ of the job $j$.
* The minimization of the Objective function (34) forces the last operations of all jobs to start globally as soon as possible.

* The boolean variable $x_{ij}^{t}$ takes the value 1 if the operation $i$ of the job $j$ starts in period $t$, with $i = 1..n_i$, $j = 1..n$, $t = 1..T$, and takes the value 0 otherwise.
* We note $M_{ij}$, $i = 1..n_i, j = 1..n$, the required machine for the operation $i$ of the job $j$.
* The minimization of the Objective function (34) forces the last operations of all jobs to start globally as soon as possible.

$$\sum_{j} \sum_{t} t.x_{n_i j}^{t} \tag{34}$$

We force each operation to start exactly once with the following set of constraints (35) to be relaxed in the Objective function.

We force each operation to start exactly once with the following set of constraints (35) to be relaxed in the Objective function.

$$(\sum_{t} x_{ij}^{t} - 1)^2, \qquad i = 1..n_i, j = 1..n \tag{35}$$

Constraints (36) forbid to have more than one operation at a time on a given machine.

Constraints (36) forbid to have more than one operation at a time on a given machine.

$$x_{ij}^{t} x_{i'j'}^{t'} = 0,$$
$$\forall (i,j,t) \cup (i',j',t') : i,i' = 1..n_i, j,j' = 1..n, (i,j) \neq (i',j'),$$
$$M_{ij} = M_{i'j'}, (t,t') \in T^2, 0 \leq t' - t < p_{ij} \tag{36}$$

Constraints (37) forbid consecutive operations to start before the previous one is finished.

Constraints (37) forbid consecutive operations to start before the previous one is finished.

$$x_{ij}^{t} x_{i+1j}^{t'} = 0, \quad i = 1..(n_i - 1), j = 1..n, (t, t') \in T^2, t + p_{ij} > t' \qquad (37)$$

$$
\begin{aligned}
&\sum_j \sum_t t.x^t_{n_i j} \\
+\lambda_1 &\sum_j \sum_i (\sum_t x^t_{ij} - 1)^2 \\
+\lambda_2 &\sum_{(i,j,t)\cup(i',j',t') \in T1} x^t_{ij} x^{t'}_{i'j'} \\
+\lambda_3 &\sum_{(i,j,t,t') \in T2} x^t_{ij} x^{t'}_{i+1j}
\end{aligned}
\tag{38}
$$

with:

$$T1 = (i,j,t) \cup (i',j',t') : i,i' = 1..n_i, j,j' = 1..n, (i,j) \neq (i',j'),$$

$$M_{ij} = M_{i'j'}, (t,t') \in T^2, 0 \leq t' - t < p_{ij}$$

$$T2 = (i,j,t,t') : i = 1..(n_i - 1), j = 1..n, (t,t') \in T^2, t + p_{ij} > t'$$

Table 6: Results on 5 instances (4 jobs, 4 machines and 4 operations per job) solved by the D-Wave Hybrid Solution ([Aggoune and Deleplanque, 2023]).

| | T=19 | T=13 | T=11 | T=9 | T=8 | T=7 | T=6 | T=5 |
|---|---|---|---|---|---|---|---|---|
| *inst1* | 41 | 30 | 25 | 26 | 25 | 24 | 21 | *n.s.e.* |
| *inst2* | 38 | 29 | 26 | 26 | 25 | 21 | 22 | *s.n.f.* |
| *inst3* | 46 | 29 | 27 | 24 | 20 | 20 | 18 | 18 |
| *inst4* | 55 | 38 | 38 | 31 | 29 | 28 | *s.n.f.* | *n.s.e.* |
| *inst5* | 42 | 28 | 25 | 21 | 21 | 20 | 19 | 18 |

# >>> From the QUBO graph to the qubit graph

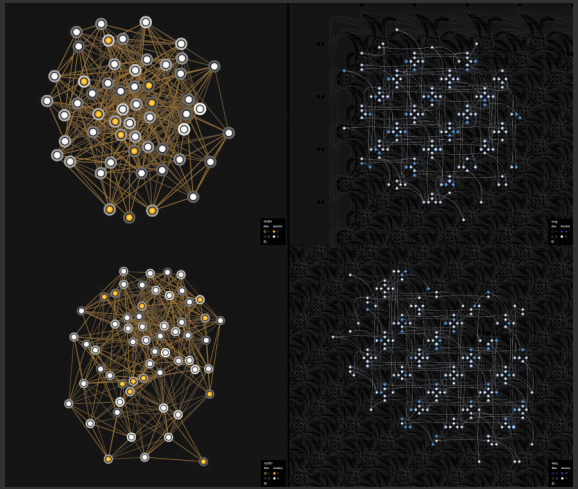


Figure 29: Embedding a QUBO Job Shop instance (left, 48 variables) in the Chimera (top right, 122 qubits) and Pegasus topology (bottom right, 208 qubits) [Aggoune and Deleplanque, 2024]

Quantum hybrid solution
for solving
the

# Capacitated Vehicle Routing Problem

by

## *clustering first - route second*

-

a work with

-

Amina El Yaagoubi (LITL), Julia Mourier (JUNIA)

-

Téo Gras, Florent Descamps (JUNIA)

-

Isabelle Lefebvre (CNRS)

-

[Deleplanque et al., 2025b]

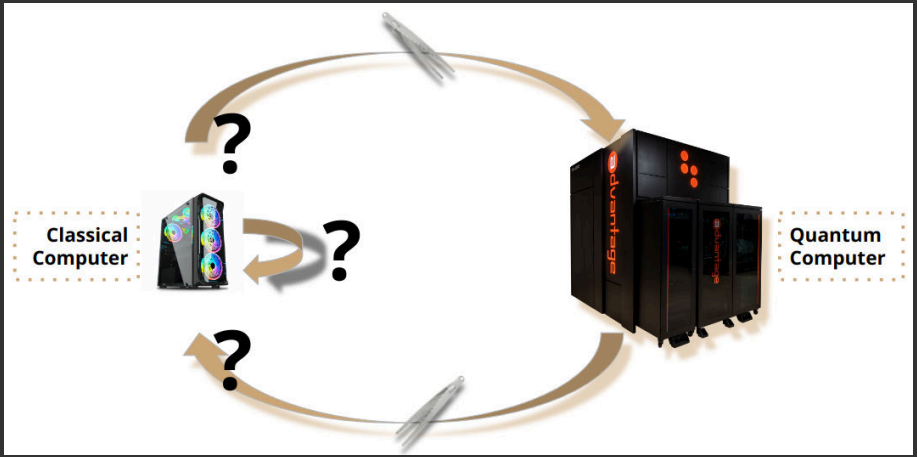



Figure 30: What is really happening?

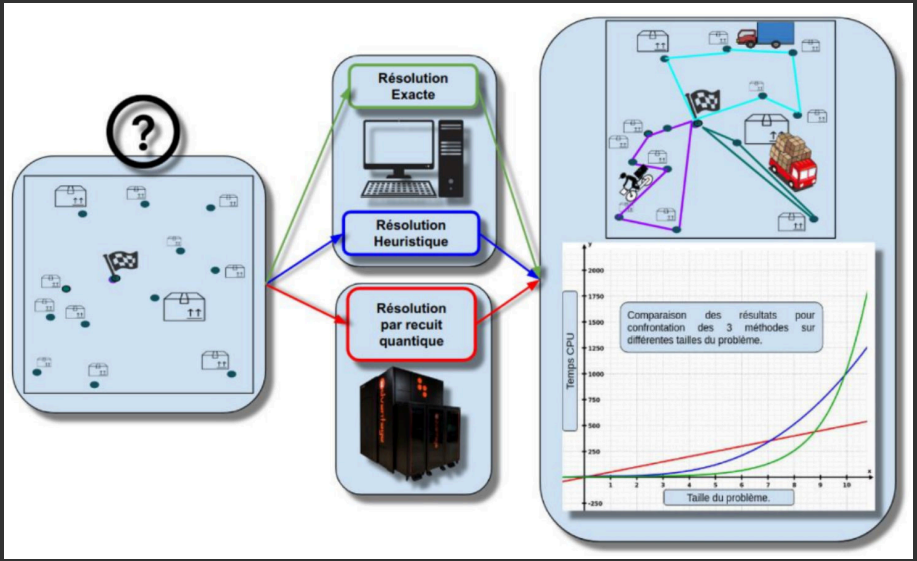



Figure 31: CVRP

Strategy: Master Clustering and Slave TSP(s) problems.

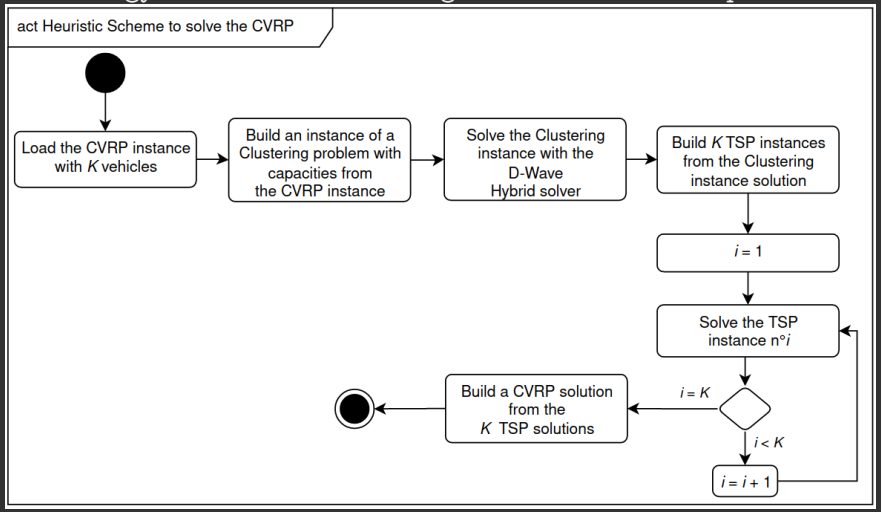



Figure 32: UML Activity Diagram: resolution process of the CVRP.

| | |
|---|---|
| $\mathcal{N}$ | The set of customers |
| $\mathcal{K}$ | The set of vehicles |
| $Q_k$ | Capacity of the vehicle $k$ |
| $u_i$ | Capacity consumption of customer $i, i \in \mathcal{N}$ |
| $c_{ij}$ | Cost for going from customer $i$ to customer $j$, $(i,j) \in \mathcal{N}^2$ |

Table 7: Clustering problem parameters.

| | |
|---|---|
| $\mathcal{N}$ | The set of customers |
| $\mathcal{K}$ | The set of vehicles |
| $Q_k$ | Capacity of the vehicle $k$ |
| $u_i$ | Capacity consumption of customer $i, i \in \mathcal{N}$ |
| $c_{ij}$ | Cost for going from customer $i$ to customer $j$, $(i,j) \in \mathcal{N}^2$ |

Table 7: Clustering problem parameters.

* To model the clustering problem with capacities, we note $x$ a decision variable vector where each element $x_{ik}, i \in \mathcal{N}, k \in \mathcal{K}$, takes a $\{0;1\}$ value such that:

$$x_{ik} = \begin{cases} 1 & \text{if the custumer } i, i \in \mathcal{N}, \text{is being taken in charge by } k, \\ 0 & \text{otherwise.} \end{cases}$$

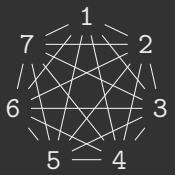
## >>> Clustering: the Objective function.

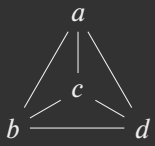
* The quadratic objective function $f_{Clus}(x)$ is defined in the expression (39).

$$f_{Clus}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{j=i+1}^{|\mathcal{N}|} \sum_{k=1}^{|\mathcal{K}|} c_{ij} x_{ik} x_{jk} \tag{39}$$

* The quadratic objective function $f_{Clus}(x)$ is defined in the expression (39).

$$f_{Clus}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{j=i+1}^{|\mathcal{N}|} \sum_{k=1}^{|\mathcal{K}|} c_{ij} x_{ik} x_{jk} \tag{39}$$



* Each cluster is related to one vehicle $k, k \in \mathcal{K}$,
* the number of satisfied customers by a vehicle is bounded by its capacity $Q_k$ in the set of Constraints (40).

$$\sum_{i=1}^{|\mathcal{N}|} u_i x_{ik} \leq Q_k \qquad \forall k \in \mathcal{K} \tag{40}$$

* Each cluster is related to one vehicle $k, k \in \mathcal{K}$,
* the number of satisfied customers by a vehicle is bounded by its capacity $Q_k$ in the set of Constraints (40).

$$\sum_{i=1}^{|\mathcal{N}|} u_i x_{ik} \leq Q_k \qquad \forall k \in \mathcal{K} \tag{40}$$

* Constraints (41) force each customer to be satisfied exactly once.

$$\sum_{k=1}^{|\mathcal{K}|} x_{ik} = 1 \qquad \forall i \in \mathcal{N} \tag{41}$$

| | |
|---|---|
| $\mathcal{N}_k$ | The set of customers assigned to the vehicle $k, k \in \mathcal{K}$ |
| $\mathcal{P}_k$ | Consecutive customers (positions) in the route $k, k \in \mathcal{K}$ |
| $Q_k$ | Capacity of the vehicle $k$ |

Table 8: TSP problem parameters.

>>> TSP: parameters and variables.

| | |
|---|---|
| $\mathcal{N}_k$ | The set of customers assigned to the vehicle $k, k \in \mathcal{K}$ |
| $\mathcal{P}_k$ | Consecutive customers (positions) in the route $k, k \in \mathcal{K}$ |
| $Q_k$ | Capacity of the vehicle $k$ |

Table 8: TSP problem parameters.

* From the clustering problem solution obtained in a first step, we create $|\mathcal{K}|$ Traveling Salesman Problem instances.
* We note $x^k$ a decision variable vector dedicated to the vehicle $k, k \in \mathcal{K}$, where each element $x^k_{ip}, i \in \mathcal{N}, p \in \mathcal{P}$, takes a $\{0;1\}$ value such that:

$$x^k_{ip} = \begin{cases} 1 & \text{if the custumer } i, i \in \mathcal{N}, \text{is visited at the } p^{th} \text{ position in the route of} \\ 0 & \text{otherwise.} \end{cases}$$

* We note $f_{TSP}$ the quadratic objective function of the TSP given in expression (42).

$$f_{TSP}(x^k) = \sum_{i=1}^{|\mathcal{N}_k|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}_k|} (c_{ij} x^k_{i\mathcal{P}_k} x^k_{j1} + \sum_{p=1}^{|\mathcal{P}_k|-1} c_{ij} x^k_{ip} x^k_{jp+1}) \tag{42}$$

* We note $f_{TSP}$ the quadratic objective function of the TSP given in expression (42).

$$f_{TSP}(x^k) = \sum_{i=1}^{|\mathcal{N}_k|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}_k|} (c_{ij} x^k_{i\mathcal{P}_k} x^k_{j1} + \sum_{p=1}^{|\mathcal{P}_k|-1} c_{ij} x^k_{ip} x^k_{jp+1}) \tag{42}$$

* We avoid subtour constraints, the use of quadratic terms allow us to consider two consecutive customers in a the vehicle route by the product $x^k_{ip} x^k_{jp+1}$.
* If both $i$ and $j$ are managed in succession, we pay the cost $c_{ij}$.
* The expression $c_{ij} x^k_{i\mathcal{P}_k} x^k_{j1}$ considers the cost of the way back to the first element $(p = 1)$ from the last one $p = \mathcal{P}_k$.

* Constraints (43) force the vehicle $k, k \in \mathcal{K}$, to satisfy exactly once all its $|\mathcal{N}_k|$ customers.

$$\sum_{p=1}^{\mathcal{P}_k} x_{ip}^k = 1 \qquad \forall i \in \mathcal{N}_k \tag{43}$$

* Constraints (43) force the vehicle $k, k \in \mathcal{K}$, to satisfy exactly once all its $|\mathcal{N}_k|$ customers.

$$\sum_{p=1}^{\mathcal{P}_k} x_{ip}^k = 1 \qquad \forall i \in \mathcal{N}_k \tag{43}$$

* Constraints (44) force the vehicle route $k, k \in \mathcal{K}$, to satisfy exactly one customer at each position of its route.

$$\sum_{i=1}^{\mathcal{N}_k} x_{ip}^k = 1 \qquad \forall p \in \mathcal{P}_k \tag{44}$$

For a right couple $(\lambda_1, \lambda_2)$, we try to find $x$ minimizing $f_{TSP}^{QUBO}(x)$ defined as follows.

$$f_{TSP}^{QUBO}(x) = f_{TSP}(x) + Cons_1(x) + Cons_2(x) \tag{45}$$

$$f_{TSP}(x) = \sum_{i=1}^{|\mathcal{N}|} \sum_{\substack{j=1 \\ i \neq j}}^{|\mathcal{N}|} (c_{ij} x_{ip} x_{j1} + \sum_{p=1}^{|\mathcal{P}|-1} c_{ij} x_{ip} x_{jp+1}) \tag{46}$$

$$Cons_1(x) = \lambda_1 \sum_{i=1}^{|\mathcal{N}|} (- \sum_{p=1}^{|\mathcal{P}|} x_{ip} + 2 \sum_{p=1}^{|\mathcal{P}|} \sum_{q>p}^{|\mathcal{P}|} x_{ip} x_{iq}) \tag{47}$$

$$Cons_2(x) = \lambda_2 \sum_{p=1}^{|\mathcal{P}|} (- \sum_{i=1}^{|\mathcal{N}|} x_{ip} + 2 \sum_{i=1}^{|\mathcal{N}|} \sum_{j>i} x_{ip} x_{jp}) \tag{48}$$

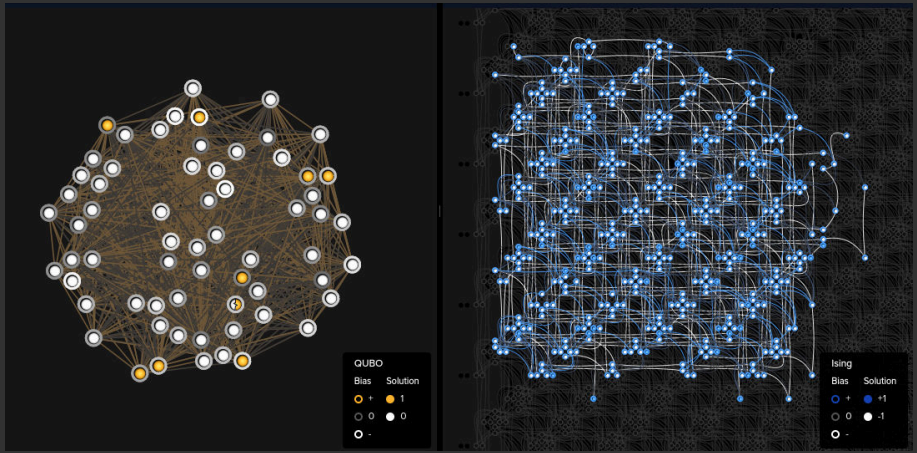



Figure 33: Instance graph and its mapping into the Pegasus topology: from 8 cities and its 64 variables to 455 qubits.

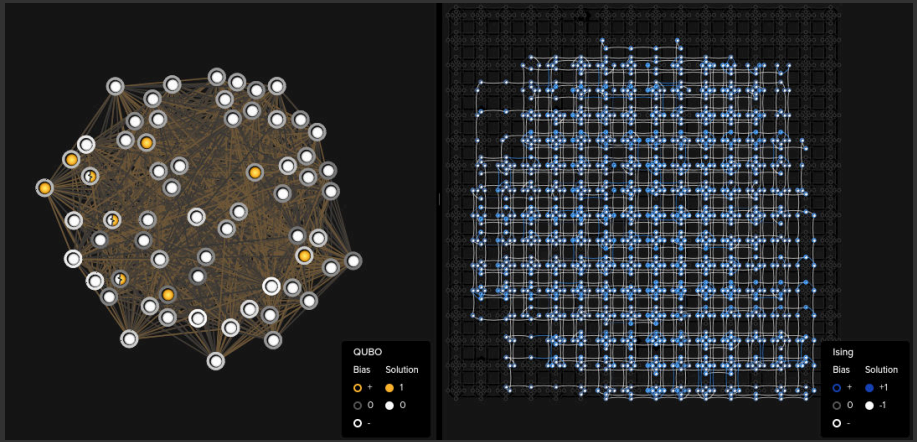



Figure 34: Instance graph and its mapping into the Chimera topology: from 8 cities and its 64 variables to 1103 qubits.

```
numberOfCars          = 6
numberOfCities        = 300
capaConsumptionMin    = 1
capaConsumptionMax    = 4
```

Some XP on “home” instances

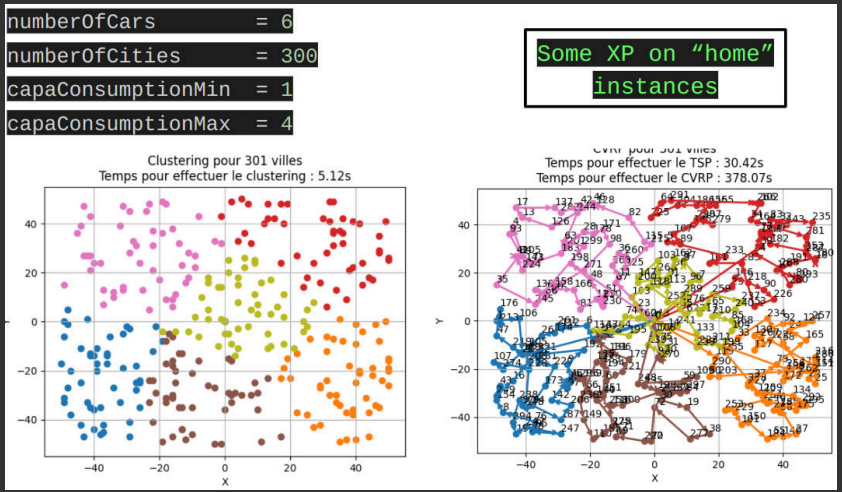



Figure 35: Left: Clustering results. Right: Multiple TSPs results.

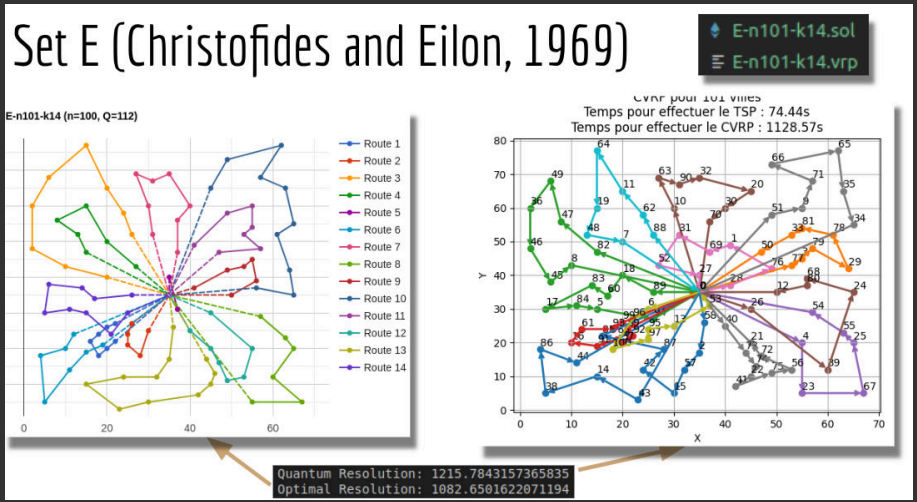



Figure 36: Left: Clustering results. Right: Multiple TSPs results.

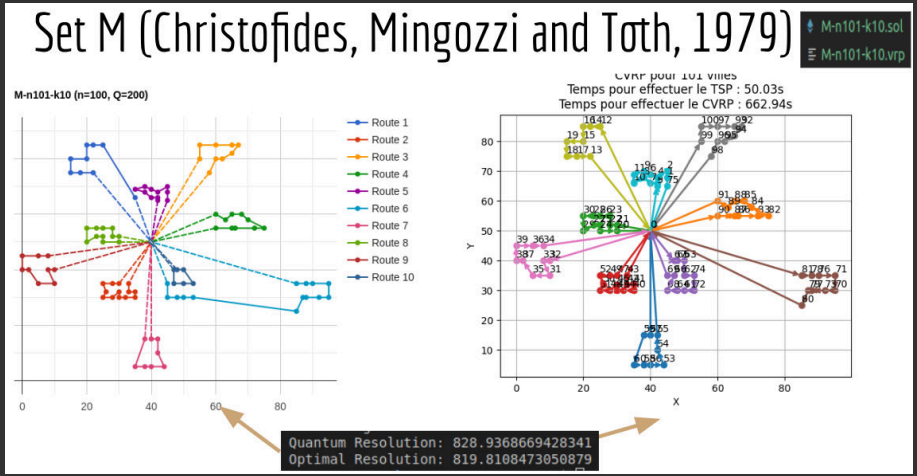



Figure 37: Left: Clustering results. Right: Multiple TSPs results.

# >>> References I




Aggoune, R. and Deleplanque, S. (2023).
A quantum annealing solution to the job shop scheduling problem.
In *Lecture Notes in Computer Science, Springer; International Conference on Computational Science and Its Applications*, pages 421-428. Springer.



Aggoune, R. and Deleplanque, S. (2024).
Addressing machine unavailability in job shop scheduling: A quantum computing approach.
Metaheuristics Part I, (LNCS 14753), Chapter 23, Springer.



Carugno, C., Ferrari Dacrema, M., and Cremonesi, P. (2022).
Evaluating the job shop scheduling problem on a d-wave quantum annealer.
*Scientific Reports*, 12(1):1-11.



Deleplanque, S., Durbec, A., and El Yaagoubi, A. (2025a).
Optimizing bin packing problem on quantum annealers: A qubo-based approach and scalability challenges.
*Optimization and Engineering*.
In press.



Deleplanque, S., El Yaagoubi, A., Gras, T., Descamps, F., Mourrier, J., and Lefebvre, I. (2025b).
Applying analog quantum computing to solve optimisation problems: case studies on max-cut and cvrp.
*International Journal of Systems Science: Operations & Logistics*, 12(1):2463530.



Farhi, E., Goldstone, J., and Gutmann, S. (2014).
A quantum approximate optimization algorithm.
*arXiv preprint arXiv:1411.4028*.



Festa, P., Pardalos, P. M., Resende, M. G., and Ribeiro, C. C. (2002).
Randomized heuristics for the max-cut problem.
*Optimization methods and software*, 17(6):1033-1058.

# >>> References II




Grover, L. K. (1996).
A fast quantum mechanical algorithm for database search.
In *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pages 212-219.



Rieffel, E. G., Venturelli, D., O'Gorman, B., Do, M. B., Prystay, E. M., and Smelyanskiy, V. N. (2015).
A case study in programming a quantum annealer for hard operational planning problems.
*Quantum Information Processing*, 14(1):1-36.



Shor, P. W. (1997).
Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer.
*SIAM Journal on Computing*, 26(5):1484-1509.



Yazdani, M., Aleti, A., Khalili, S. M., and Jolai, F. (2017).
Optimizing the sum of maximum earliness and tardiness of the job shop scheduling problem.
*Computers & Industrial Engineering*, 107:12-24.